INF1008-Assignment 2

Team ID:

Team Members: Jerome Goh, Ang Jin Heng, Brendan Tjung, Vernell Lim Xi, Pang Khae Yong Darryl, Ryan Ang Rui Heng

Student ID: 2401012, 2400937, 2401765, 2402271, 240116, 2400522

Student Email: 2401012@sit.singaporetech.edu.sg, 2400937@sit.singaporetech.edu.sg, 2401765@sit.singaporetech.edu.sg, 2402271@sit.singaporetech.edu.sg, 2401161@sit.singaporetech.edu.sg, 2400522@sit.singaporetech.edu.sg

Introduction:

This report explores the implementation of a multi-level linked list with other helper data structures for an organisation chart and researches other linked list structures. The **list** will serve as the primary data structure.

The following Artificial Intelligence (AI) chatbots were being used with the respective lists:

- Circular Linked List (Github Copilot)
- Doubly Circular Linked List (Gemini)
- Skip List (ChatGPT)
- Multi-Level Linked List (DeepSeek)

Each AI tool contributed to different aspects of the implementation and research of the different lists. This report details the different functions of each list and the creation & process of the application.

Table Of Contents

1) Circular Linked List

A circular linked list (Figure 1.1) is a data structure where the last node connects back to the first node, forming a circle/loop. Like a linked list, each node will have a “next” pointer. However, unlike the linked list, the “next” of the last node will not point to null; instead, it will point back to the first node. Circular linked lists can grow or shrink dynamically as elements are added or removed. Circular linked lists have several applications, such as resource allocation: circular linked lists are used to implement round-robin scheduling, music and video playback: circular linked lists can be used to implement continuous playback of songs or videos, buffer management: circular linked lists can be used to implement circular buffers, which are fixed-size data structures that hold a limited number of elements and overwrite old elements when full and circular queues.

In this report, using Copilot, the team will analyse the circular linked list and its advantages and disadvantages and evaluate its response.

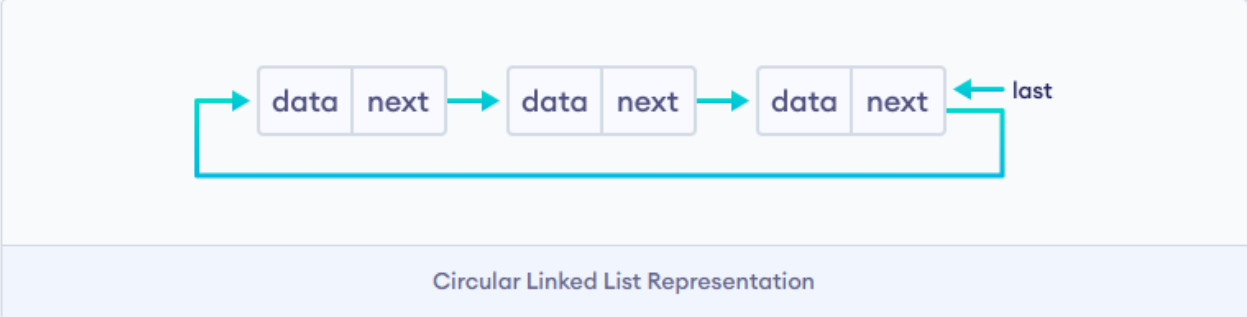


Figure 1.1

Interaction with Copilot:

Question 1: Describe a circular linked list

Question 2: Advantages and disadvantages of circular linked list

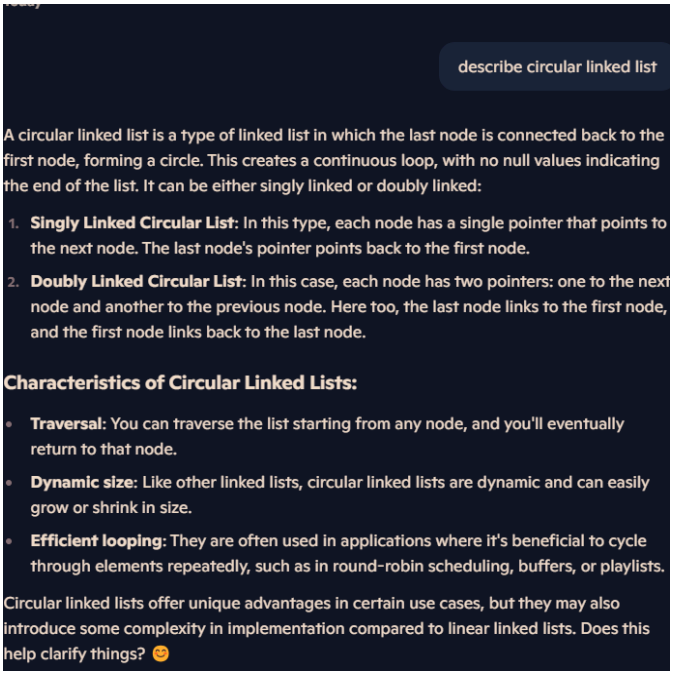


Figure 1.2

Figure 1.3

Question 3: When should I use a circular linked list over other structures
Question 4: Time and space complexity of circular linked list

Circular linked lists excel in specific scenarios where their unique features provide clear advantages. Here are some situations where they stand out compared to other structures:

1. Round-Robin Scheduling

- Circular linked lists are perfect for round-robin algorithms used in operating systems for task scheduling. Each process or task can be represented as a node, and moving to the next task is seamless due to the circular structure.

2. Buffer Management

- In cases like buffering in streaming or data transfer (e.g., audio, video, or real-time communication), circular linked lists can manage cyclic data effectively, ensuring that memory is reused efficiently.

3. Multiple Cycles Through Data

- When you need to cycle through data repeatedly—like in a playlist or a game with players taking turns—a circular linked list simplifies the implementation since there's no explicit end to the structure.

4. Implementing Queues

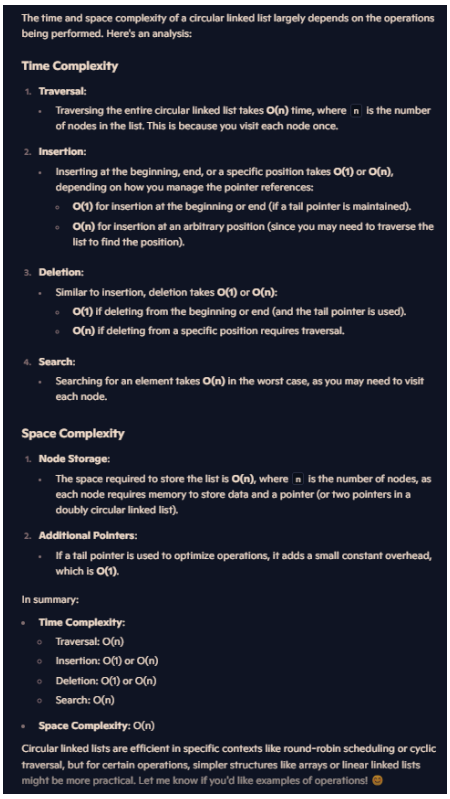
- Circular queues, used in scenarios like printer job queues or task queues, can benefit from a circular linked list because it helps manage the queue without the need to resize or shift elements.

5. Applications Where Dynamic Size is Needed

- If you need the flexibility of dynamic size, and the cyclic nature is an advantage (e.g., in simulations or algorithms like Josephus problem), circular linked lists are a better choice.

However, for simpler use cases or those requiring frequent random access to elements, other structures like arrays or linear linked lists might be more appropriate. Do you have a specific application in mind where you're considering using a circular linked list? 🤓

Figure 1.5



The time and space complexity of a circular linked list largely depends on the operations being performed. Here's an analysis:

Time Complexity

1. **Traversal:**
   - Traversing the entire circular linked list takes **O(n)** time, where `n` is the number of nodes in the list. This is because you visit each node once.
2. **Insertion:**
   - Inserting at the beginning, end, or a specific position takes **O(1)** or **O(n)**, depending on how you manage the pointer references:
     - **O(1)** for insertion at the beginning or end (if a tail pointer is maintained).
     - **O(n)** for insertion at an arbitrary position (since you may need to traverse the list to find the position).
3. **Deletion:**
   - Similar to insertion, deletion takes **O(1)** or **O(n)**:
     - **O(1)** if deleting from the beginning or end (and the tail pointer is used).
     - **O(n)** if deleting from a specific position requires traversal.
4. **Search:**
   - Searching for an element takes **O(n)** in the worst case, as you may need to visit each node.

Space Complexity

1. **Node Storage:**
   - The space required to store the list is **O(n)**, where `n` is the number of nodes, as each node requires memory to store data and a pointer (or two pointers in a doubly circular linked list).
2. **Additional Pointers:**
   - If a tail pointer is used to optimize operations, it adds a small constant overhead, which is **O(1)**.

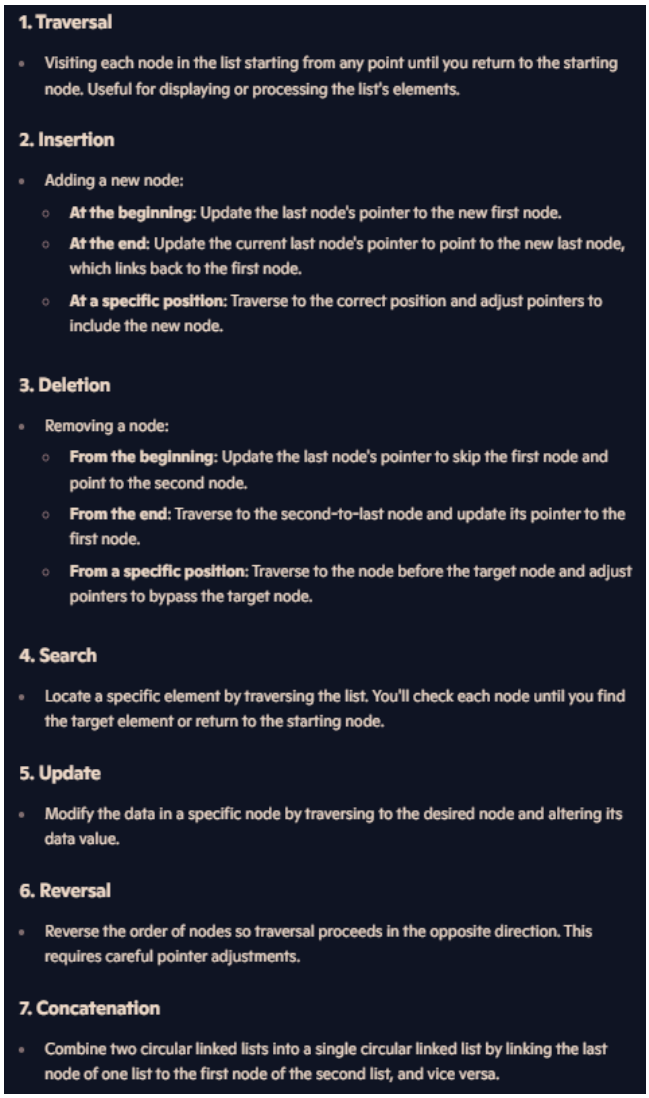
In summary:

- **Time Complexity:**
  - Traversal: O(n)
  - Insertion: O(1) or O(n)
  - Deletion: O(1) or O(n)
  - Search: O(n)
- **Space Complexity:** O(n)

Circular linked lists are efficient in specific contexts like round-robin scheduling or cyclic traversal, but for certain operations, simpler structures like arrays or linear linked lists might be more practical. Let me know if you'd like examples of operations! 😊

Figure 1.6

Question 5: What are the different operations for a circular linked list



1. Traversal

- Visiting each node in the list starting from any point until you return to the starting node. Useful for displaying or processing the list's elements.

2. Insertion

- Adding a new node:
  - **At the beginning:** Update the last node's pointer to the new first node.
  - **At the end:** Update the current last node's pointer to point to the new last node, which links back to the first node.
  - **At a specific position:** Traverse to the correct position and adjust pointers to include the new node.

3. Deletion

- Removing a node:
  - **From the beginning:** Update the last node's pointer to skip the first node and point to the second node.
  - **From the end:** Traverse to the second-to-last node and update its pointer to the first node.
  - **From a specific position:** Traverse to the node before the target node and adjust pointers to bypass the target node.

4. Search

- Locate a specific element by traversing the list. You'll check each node until you find the target element or return to the starting node.

5. Update

- Modify the data in a specific node by traversing to the desired node and altering its data value.

6. Reversal

- Reverse the order of nodes so traversal proceeds in the opposite direction. This requires careful pointer adjustments.

7. Concatenation

- Combine two circular linked lists into a single circular linked list by linking the last node of one list to the first node of the second list, and vice versa.

Figure 1.6

Evaluating the response from Copilot:

Question 1: Describe a circular linked list (Figure 1.2)

Accuracy	Copilot's response to the question is accurate. Based on references to Geeksforgeeks, the explanation is accurate in explaining the structure of a circular linked list. The last pointer does not point to a null value but instead points to the first node, creating a continuous loop.
Completeness	Copilot is complete with its response to the question as it talks about the structure of circular linked lists, and it even went one step further to talk about double circular linked lists and the structure of double circular linked lists. Copilot also briefly mentioned the characteristics of a circular linked list
Clarity	Copilot clearly explains the structure of circular linked lists, which is straightforward to understand.
Relevance	The response is relevant to the question without veering off the question and only providing relevant data.

Question 2: Advantages and disadvantages of circular linked list (Figure 1.3)

Accuracy	Copilot is accurate in its response to the question. Based on references to Geeksforgeeks, the explanation is accurate in explaining the advantages and disadvantages of circular linked lists. The advantages of the circular linked list mentioned were efficient traversal, no null nodes, dynamic size, and easier queue implementation. The disadvantages of circular linked lists mentioned were the more complex implementation, difficult traversal termination, and wasted resources. These all agree with references to Geeksforgeeks.
Completeness	Copilot is complete with its response to the question as it talks about the advantages and disadvantages and explains what each advantage and disadvantage does.
Clarity	Copilot clearly explains the advantages and disadvantages of circular linked lists and is straightforward to understand.
Relevance	The response is relevant to the question did not veer off the question and provided a relevant answer to the question.

Question 3: When should I use a circular linked list over other structures (Figure 1.4)

Accuracy	Copilot is accurate in its response to the question. Based on references to Geeksforgeeks, the explanation is accurate in explaining when one should use a circular linked list over other structures. It also came with

	a quick explanation of the different applications of circular linked lists.
Completeness	Copilot is complete with its response to the question as it mentions the different cases in which circular linked lists are applicable. Applications mentioned are round-robin scheduling, buffer management, multiple cycles through data, implementing queues, and applications where dynamic size is needed.
Clarity	Copilot's response to the question is partially clear. However, it can be clearer when explaining the application of circular linked lists. For example, when the copilot talks about implementing queues in circular linked lists, it can explain more clearly why it is better than just mentioning the scenario.
Relevance	The response is relevant to the question, does not veer off the question and provides relevant answers to the question.

Question 4: Time and space complexity of circular linked list (Figure 1.5)

Accuracy	Copilot is accurate in its response to the question. Based on references to Geeksforgeeks, the explanation is accurate in explaining the different time complexities of the different operations that can be performed in a circular linked list.
Completeness	Copilot is complete with its response to the question as it talks about the different time complexities of different operations that can be performed in a circular linked list. Copilot even went one step further to explain the different cases in each operation. For example, in insertion, it can be O(1) or O(n). Copilot also mentioned the space complexities for node storage and if additional pointers were to be used.
Clarity	Copilot is clear with its response to the question as it gave the time complexity of all the different operations that can be done with circular linked lists and also gave an overall summary of the time complexities.
Relevance	The response is relevant to the question, does not veer off the question, and provides relevant answers to the question.

Question 5: What are the different operations for a circular linked list (Figure 1.6)

Accuracy	Copilot is accurate in its response to the question. Based on references to Geeksforgeeks, the explanation is accurate in mentioning the different operations. The operations mentioned are traversal, insertion, deletion, search, update, reversal, and concatenation.

Completeness	Copilot is partially complete with its response to the question. Copilot only mentioned traversal, insertion, deletion, search, update, reversal, and concatenation but missed out on more advanced functions like splitting, length calculation, and detecting loops.
Clarity	Copilot is clear with its response to the question as it mentions the operations and gives a brief but clear explanation of the operation, making it straightforward to understand
Relevance	The response is relevant to the question, does not veer off the question, and provides relevant answers to the question.

2) Doubly Circular Linked List

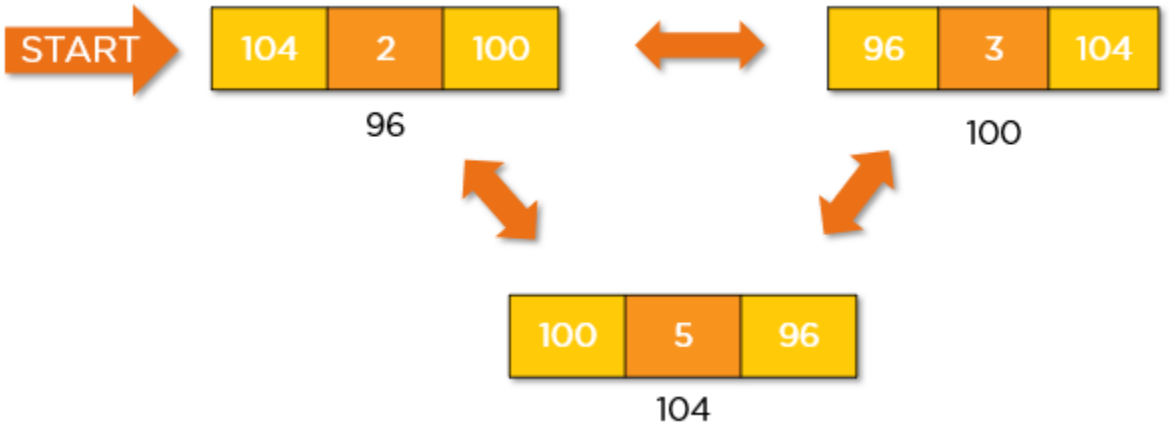


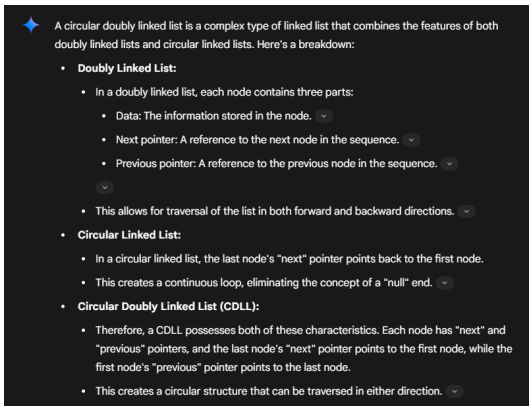
Figure 2.1

A circular doubly linked list is a mixture of a doubly linked list and a circular linked list. In a doubly circular, the first and last nodes are linked together which will then form a circle. Just like the doubly linked list, it has an extra pointer called the previous pointer. This type of linked list is a bi-directional list.

Having a doubly circular linked list allows for quick and easy traversal of the entire list without the need for special case handling. With its circular nature it could move back and forth between the first and the last node easily. Unlike the doubly linked list where the first and last nodes are not linked together.

Interaction with Gemini:

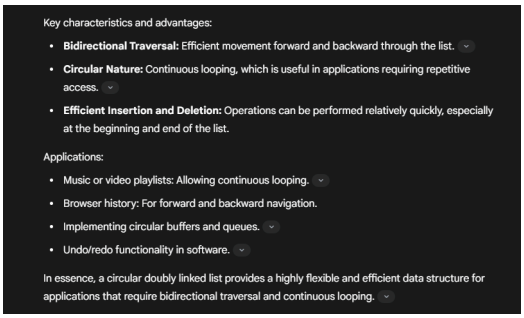
Question 1: What is a doubly circular linked list



A circular doubly linked list is a complex type of linked list that combines the features of both doubly linked lists and circular linked lists. Here's a breakdown:

- **Doubly Linked List:**
  - In a doubly linked list, each node contains three parts:
    - Data: The information stored in the node.
    - Next pointer: A reference to the next node in the sequence.
    - Previous pointer: A reference to the previous node in the sequence.
  - This allows for traversal of the list in both forward and backward directions.
- **Circular Linked List:**
  - In a circular linked list, the last node's "next" pointer points back to the first node.
  - This creates a continuous loop, eliminating the concept of a "null" end.
- **Circular Doubly Linked List (CDLL):**
  - Therefore, a CDLL possesses both of these characteristics. Each node has "next" and "previous" pointers, and the last node's "next" pointer points to the first node, while the first node's "previous" pointer points to the last node.
  - This creates a circular structure that can be traversed in either direction.

Figure 2.2



Key characteristics and advantages:

- **Bidirectional Traversal:** Efficient movement forward and backward through the list.
- **Circular Nature:** Continuous looping, which is useful in applications requiring repetitive access.
- **Efficient Insertion and Deletion:** Operations can be performed relatively quickly, especially at the beginning and end of the list.

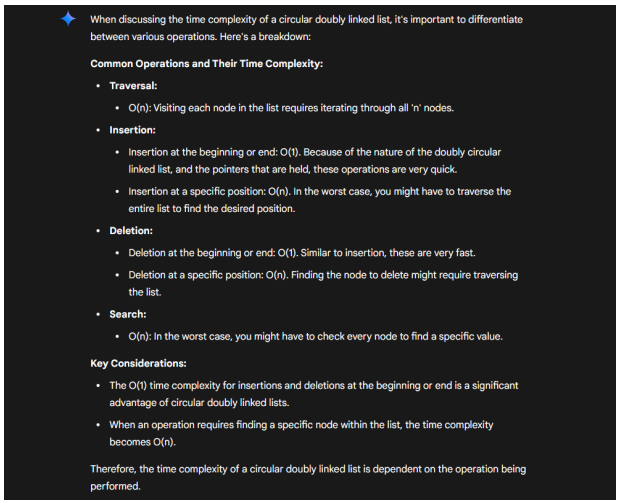
Applications:

- Music or video playlists: Allowing continuous looping.
- Browser history: For forward and backward navigation.
- Implementing circular buffers and queues.
- Undo/redo functionality in software.

In essence, a circular doubly linked list provides a highly flexible and efficient data structure for applications that require bidirectional traversal and continuous looping.

Figure 2.3

Question 2: What is the Time Complexity of a Doubly Circular Linked List?



When discussing the time complexity of a circular doubly linked list, it's important to differentiate between various operations. Here's a breakdown:

Common Operations and Their Time Complexity:

- **Traversal:**
  - O(n): Visiting each node in the list requires iterating through all 'n' nodes.
- **Insertion:**
  - Insertion at the beginning or end: O(1). Because of the nature of the doubly circular linked list, and the pointers that are held, these operations are very quick.
  - Insertion at a specific position: O(n). In the worst case, you might have to traverse the entire list to find the desired position.
- **Deletion:**
  - Deletion at the beginning or end: O(1). Similar to insertion, these are very fast.
  - Deletion at a specific position: O(n). Finding the node to delete might require traversing the list.
- **Search:**
  - O(n): In the worst case, you might have to check every node to find a specific value.

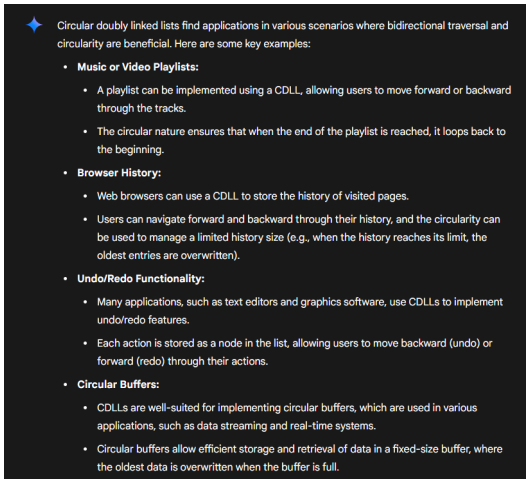
Key Considerations:

- The O(1) time complexity for insertions and deletions at the beginning or end is a significant advantage of circular doubly linked lists.
- When an operation requires finding a specific node within the list, the time complexity becomes O(n).

Therefore, the time complexity of a circular doubly linked list is dependent on the operation being performed.

Figure 2.4

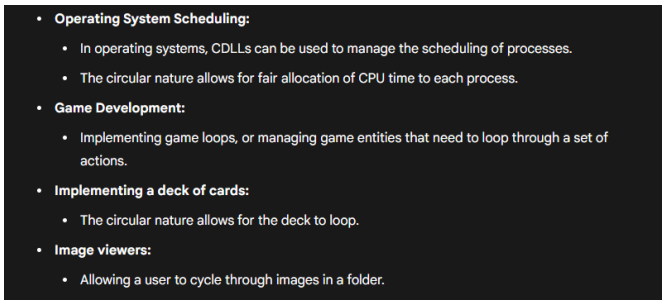
Question 3: What are some applications of it?



Circular doubly linked lists find applications in various scenarios where bidirectional traversal and circularity are beneficial. Here are some key examples:

- **Music or Video Playlists:**
  - A playlist can be implemented using a CDLL, allowing users to move forward or backward through the tracks.
  - The circular nature ensures that when the end of the playlist is reached, it loops back to the beginning.
- **Browser History:**
  - Web browsers can use a CDLL to store the history of visited pages.
  - Users can navigate forward and backward through their history, and the circularity can be used to manage a limited history size (e.g., when the history reaches its limit, the oldest entries are overwritten).
- **Undo/Redo Functionality:**
  - Many applications, such as text editors and graphics software, use CDLLs to implement undo/redo features.
  - Each action is stored as a node in the list, allowing users to move backward (undo) or forward (redo) through their actions.
- **Circular Buffers:**
  - CDLLs are well-suited for implementing circular buffers, which are used in various applications, such as data streaming and real-time systems.
  - Circular buffers allow efficient storage and retrieval of data in a fixed-size buffer, where the oldest data is overwritten when the buffer is full.

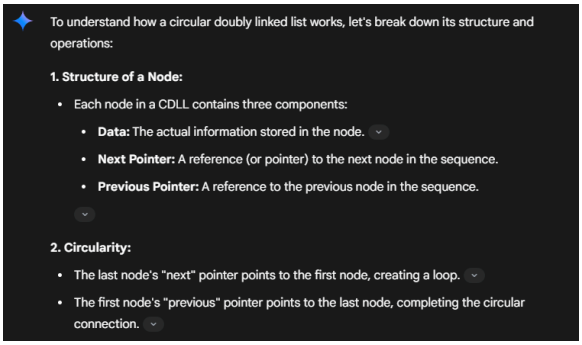
Figure 2.5



- **Operating System Scheduling:**
  - In operating systems, CDLLs can be used to manage the scheduling of processes.
  - The circular nature allows for fair allocation of CPU time to each process.
- **Game Development:**
  - Implementing game loops, or managing game entities that need to loop through a set of actions.
- **Implementing a deck of cards:**
  - The circular nature allows for the deck to loop.
- **Image viewers:**
  - Allowing a user to cycle through images in a folder.

Figure 2.6

Question 4: How does a doubly circular linked list work?



To understand how a circular doubly linked list works, let's break down its structure and operations:

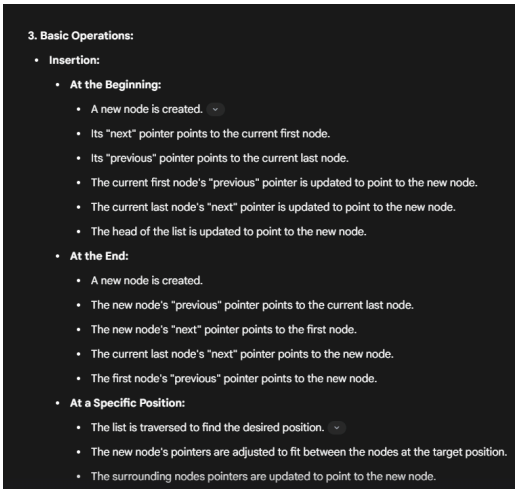
1. Structure of a Node:

- Each node in a CDLL contains three components:
  - **Data:** The actual information stored in the node.
  - **Next Pointer:** A reference (or pointer) to the next node in the sequence.
  - **Previous Pointer:** A reference to the previous node in the sequence.

2. Circularity:

- The last node's "next" pointer points to the first node, creating a loop.
- The first node's "previous" pointer points to the last node, completing the circular connection.

Figure 2.7



3. Basic Operations:

- **Insertion:**
  - **At the Beginning:**
    - A new node is created.
    - Its "next" pointer points to the current first node.
    - Its "previous" pointer points to the current last node.
    - The current first node's "previous" pointer is updated to point to the new node.
    - The current last node's "next" pointer is updated to point to the new node.
    - The head of the list is updated to point to the new node.
  - **At the End:**
    - A new node is created.
    - The new node's "previous" pointer points to the current last node.
    - The new node's "next" pointer points to the first node.
    - The current last node's "next" pointer points to the new node.
    - The first node's "previous" pointer points to the new node.
  - **At a Specific Position:**
    - The list is traversed to find the desired position.
    - The new node's pointers are adjusted to fit between the nodes at the target position.
    - The surrounding nodes pointers are updated to point to the new node.

Figure 2.8

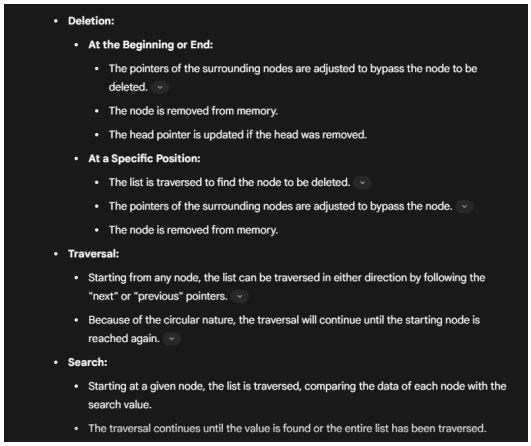


Figure 2.9

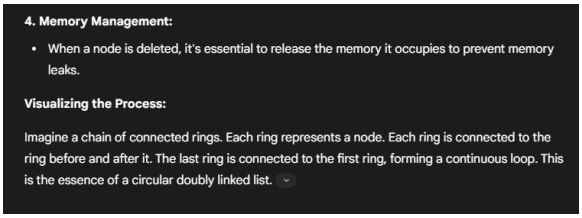


Figure 2.10

Evaluating the response from Gemini:

Question 1: What is a doubly circular linked list?

Accuracy	Gemini's result is accurate and it shows which website it cited from. Based on GeeksforGeek and WsCube, the information provided by Gemini is the same.
Completeness	The information provided by Gemini covers the main point of what represents a doubly circular linked list. It also briefly explains the characteristics and advantages of a doubly circular linked list and provides some examples of usage of the doubly circular linked list.
Clarity	The way Gemini explains the link between a doubly linked list and a circular linked list is very clear and concise.
Relevance	The answer it provides does not cover a very in-depth explanation but does provide a basic topic that covers the question

Question 2: What is the Time Complexity of a Doubly Circular Linked List?

Accuracy	Comparing results from websites like Geeksforgeek and Stackoverflow, the result generated by Gemini is the same.
Completeness	The information Gemini generated provided an explanation that covers the basics of the time complexity. It also covers the different operations and provides a simple explanation as to why it could achieve that time complexity
Clarity	The way Gemini explain it very clearly and easy to understand

Relevance	The answer covered all the necessary topics and did not go off-topic from the question.

Question 3: What are some applications of it?

Accuracy	Results generated are fact-checked with Google search and from websites used by many. The results generated are trustworthy and reliable.
Completeness	It provides a huge number of example that uses doubly circular linked lists in the algorithm
Clarity	Gemini not only provides examples of algorithms that use this linked list. It also explains how the linked list is used in each example.
Relevance	Gemini managed to cover all related topic links to the question but also cover explanations on how that explanation works

Question 4: How does a doubly circular linked list work?

Accuracy	Results generated are all pulled from reliable websites like wscubetech.com and Geeksforgeek. Gemini would also point out which points are gathered from which website.
Completeness	Gemini will break down the information it attains into the different core components and operations with a basic explanation of how the linked list works in each component
Clarity	The explanation provided is very detailed and clear. It is easy to understand the explanation of how doubly circular linked lists work in each application
Relevance	The search results are all relevant to the question asked. But it provides extra information to help readers visualize how the linked list works.

Overall Gemini chatbot is quite reliable as it will provide where it gets the result from. Gemini also has a counter-check feature where it re-analyses the response it gives and compares it with Google search results.

3) Skip List

A Skip List is a data structure that facilitates quick search, insertion, and deletion operations on a sorted sequence of elements which has an average time complexity of O(log n) and the worst case would be O(n).

It is sorted like an array in layers but with a linked list functionality and for each layer, there will be a skip on each node. The first layer will be a normal lane while the subsequent layer will be express lanes.

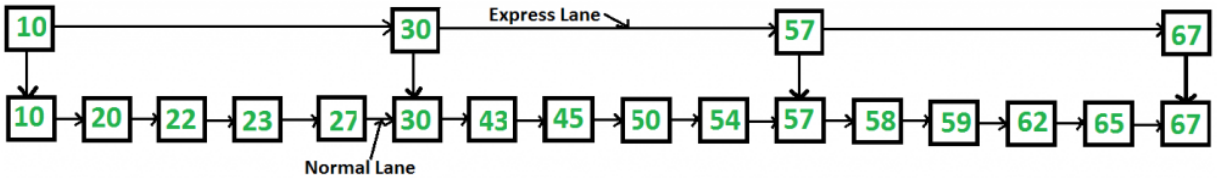


Figure 3.1 (Skip List geeks for geeks)

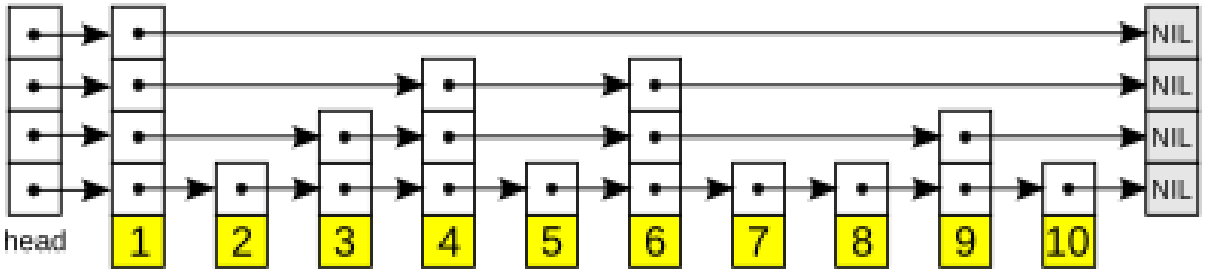


Figure 3.2 (Skip List Wikipedia)

In this case, the highest level is the layer at the top which would be the express lane. The way that Skip List would work is by checking from the highest layer and going through the nodes in each layer till it reaches the bottom layer.

Using ChatGPT to research on Skip List these are the results it gave.

Interaction with ChatGPT:

Question 1: Describe skip list

describe skip list

A **Skip List** is a probabilistic data structure designed for efficient searching, insertion, and deletion operations within a sorted list. It is essentially a hierarchy of linked lists, with the lowest level containing all elements, and each higher level acting as an "express lane," allowing faster traversal.

How Skip Lists Work:

- **Multiple Levels:** A skip list contains multiple linked lists stacked vertically:
  - **Bottom Level:** Contains **all elements** in sorted order.
  - **Higher levels** are "express lanes," containing fewer nodes and enabling faster searches.

Visual Representation:

rust Copy Edit

```
Level 3:   5 --------------> 30 ------------------------> 60
           ↓                 ↓                           ↓
Level 2:   5 -----> 15 ----> 30 -----> 45 ------------> 60
           ↓        ↓        ↓        ↓                 ↓
Level 1:   5 -> 10 ->15 --> 30 --> 40 --> 45 -----> 55 -> 60 -> 70
```

Figure 3.3

How Skip List Works:

- **Search**:
  - Begins at the highest level.
  - Moves horizontally until the next node is greater than the target, then moves down a level.
  - Continues until the target is found or confirmed absent.
- **Insertion**:
  - Similar to search, finds appropriate position at the lowest level.
  - Inserts the new node and randomly decides how many higher levels this node should occupy (coin-flip probability).
  - Links the new node across multiple levels.
- **Deletion**:
  - Finds and removes nodes from each level they're present in.

Complexity:

- **Average**: $O(\log n)$ (efficient for typical scenarios)
- **Worst-case**: $O(n)$, rare due to randomization.

Figure 3.4

Question 2: What are the advantages and disadvantages of skip list

Applications:

- Efficient indexing and searching in databases.
- Memory-efficient in-memory storage (e.g., Redis databases).
- Routing and navigation tasks in networking and distributed systems.

Would you like a detailed example or implementation of a Skip List in code?

Figure 3.5

What is the advantages and disadvantages of skip list

Here's a clear breakdown of the **advantages** and **disadvantages** of **Skip Lists**:

✅ Advantages of Skip Lists:

1. **Fast Operations**:
   - **Average complexity** for search, insert, and delete operations is **O(log n)**.
   - Efficient handling of large datasets.
2. **Simple Implementation**
   - Easier to implement than balanced trees (AVL, Red-Black).
   - Requires fewer complex rotation/rebalancing operations.
3. **Dynamic Structure**
   - Easily expandable.
   - Handles insertions/deletions dynamically without extensive restructuring.

Figure 3.6

4. **Memory Efficiency**
   - Generally uses less overhead than balanced trees.
   - Adjustable level height for memory/performance trade-off.
5. **Probabilistic Balancing**
   - Randomization helps to ensure consistent performance.
   - Rarely experiences worst-case scenarios.

How Does it Work? (Simplified Example)

Sorted List: `[10, 20, 30, 40, 50, 60]`

rust Copy Edit

```
Top Level:   10 ------------------> 50
             ↓                      ↓
Mid Level:   10 --------> 30 ----->50
             ↓           ↓         ↓
Bottom Level:10 --> 20 --> 30 --> 40 --> 50 --> 60
```

Figure 3.7

- The **top level** is an express lane to skip large segments.
- **Middle levels** provide shortcuts to jump quickly through the list.
- The **lowest level** contains every element.

Basic Operations:

- **Search**:
  - Start from the top-left node.
  - Move forward; if the next node is greater than target, move down.
  - Continue until found or reach the lowest level without finding the target.
- **Insert**:
  - Determine random levels for the new node.
  - Insert at each appropriate level based on random choice.
- **Deletion**:
  - Perform search to locate the node.
  - Adjust pointers at all levels where the node exists.

Figure 3.8

Complexity Table:

Operation	Average Case	Worst Case
Search	**O(log n)**	O(n)
Insert	**O(log n)**	O(n)
Delete	**O(log n)**	O(n)

Example Use-cases:

- **Database indexing** (fast queries and updates)
- **In-memory caches** (e.g., Redis sorted sets)
- **Filesystem indexing**
- High-speed dictionary lookups

Figure 3.9

Question 3: List some Skip List examples in the real world

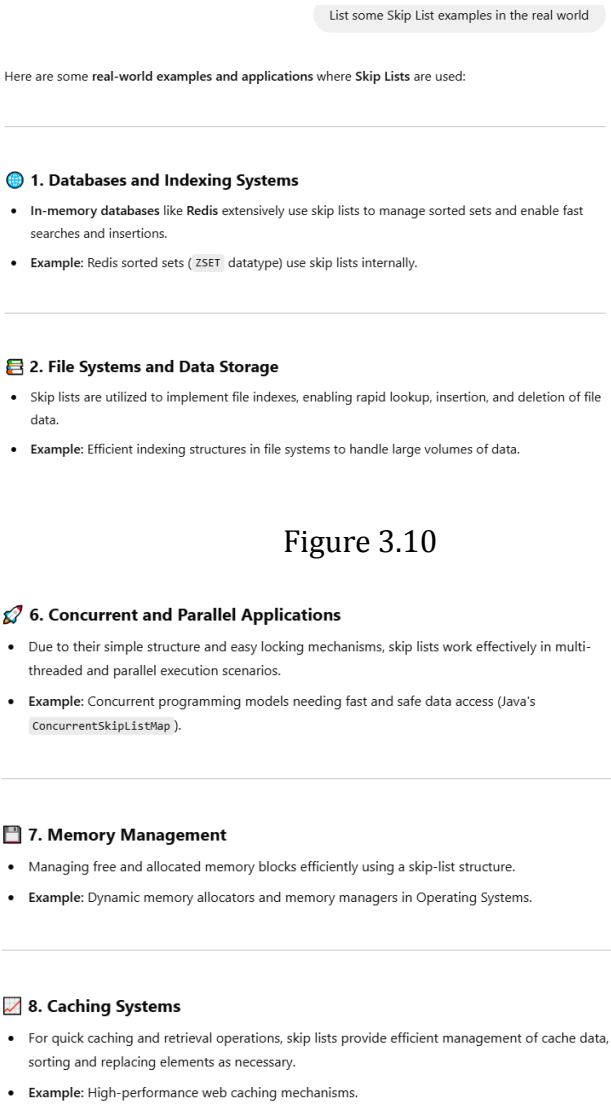


Figure 3.10

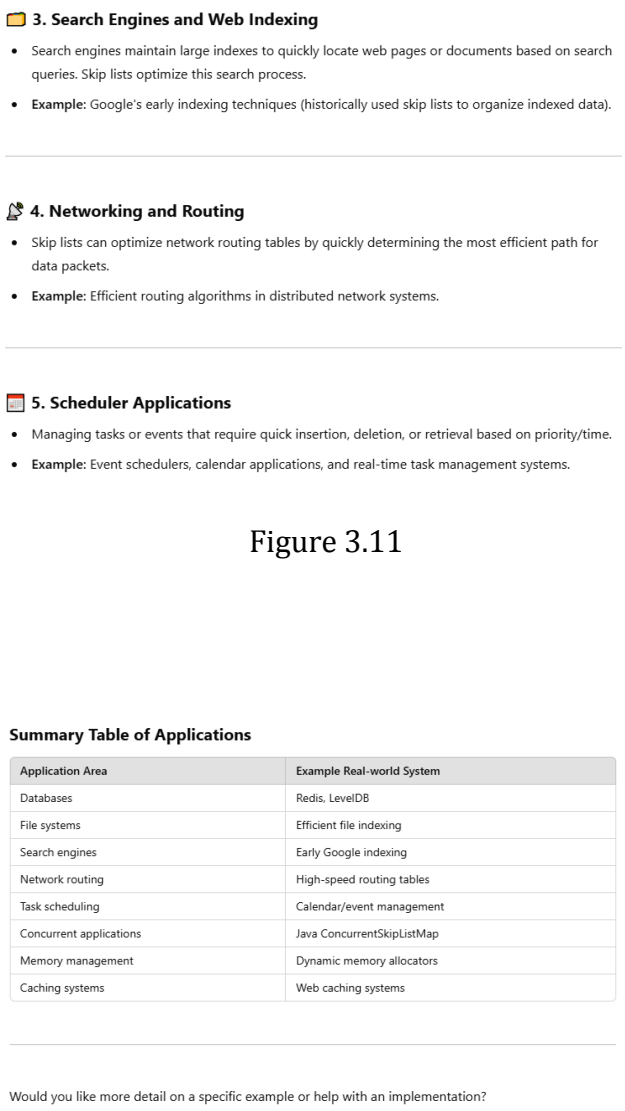


Figure 3.11

Figure 3.12

Figure 3.13

Evaluating the response from ChatGPT:

Question 1: Describe skip list

Accuracy	The results given by GPT in Figure 3.3, Figure 3.4 and Figure 3.5 are very accurate, by comparison to Geeksforgeeks it shows the same concept and produces the same structure for the skip list.
Completeness	It is complete as it describes what Skip List is, how it works, its time

	complexity and its applications of Skip List.
Clarity	It is clear and concise in its text and results with details explaining each answer.
Relevance	It is relevant to the question asked and does not divert to any other topic.

Question 2: What are the advantages and disadvantages of skip list

Accuracy	Compared to Geeksforgeeks, it matches the advantages with the results given so it is accurate.
Completeness	It missed out on the disadvantages part so it is only half complete
Clarity	The results are clear and simple to understand.
Relevance	It went off-topic after listing out the advantages and started to give results on how Skip List works.

Question 3: List some Skip List examples in the real world

Accuracy	Comparing the results to the online searches and posts by other people, it is quite accurate in the examples.
Completeness	It is quite complete looking at the online posts.
Clarity	It is clear in the list of examples but for the details in the example, it will need another prompt to get the details from ChatGPT.
Relevance	It is relevant to the topic asked in the prompt.

Summary:

Through the interaction with ChatGPT, the information given by it on how Skip List works matches with the information in Wikipedia and Geeksforgeeks where it starts the operation from the highest layer before making its way down to the lower layers. In the case of getting information on advantages and disadvantages, it is not so reliable as it would only give the advantages and leave the disadvantages out and it does give a few more advantages not mentioned by Geeksforgeeks. Overall, the chatbot breaks down the information and explains it quite clearly with a simple visual representation of how Skip List works and explains the steps. The chatbot will stay on the topic of what question was prompted to it and it will give detailed information on the subject and answer the question.

4) Multi-Level Linked List

A multi-levelled linked list is a two-dimensional data structure composed of several linked lists. It consists of several nodes where the first node is represented as the head, and the nodes contain at least 3 parts each, being interconnected using pointers:

1. *Pointer to next node* (accessing nodes on the same level)
2. *Pointer to a child node* (accessing nodes on the level below)
3. *Data* (store information)

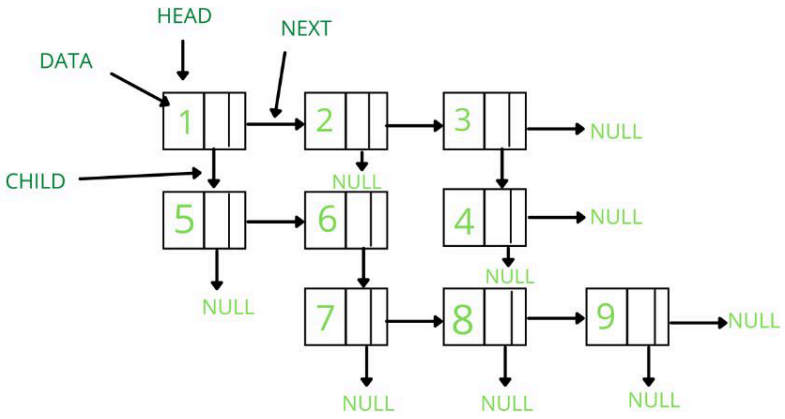
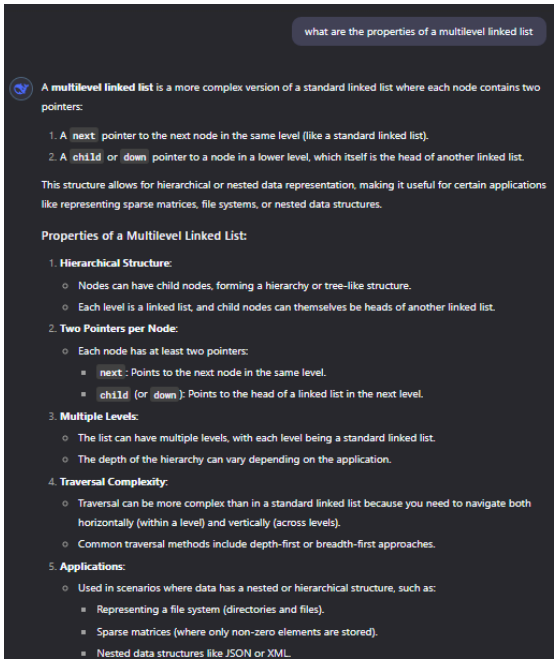


Figure 4.1

Using DeepSeek, this report will analyse the properties of the multi-level linked list and its functional capabilities.

Interaction with DeepSeek:

Question 1: What are the properties of a multi-level linked list?



what are the properties of a multilevel linked list

A **multilevel linked list** is a more complex version of a standard linked list where each node contains two pointers:

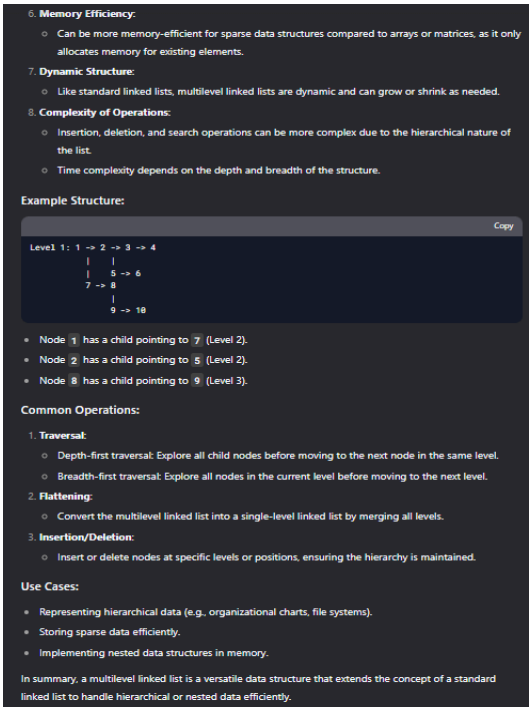
1. A `next` pointer to the next node in the same level (like a standard linked list).
2. A `child` or `down` pointer to a node in a lower level, which itself is the head of another linked list.

This structure allows for hierarchical or nested data representation, making it useful for certain applications like representing sparse matrices, file systems, or nested data structures.

Properties of a Multilevel Linked List:

1. **Hierarchical Structure**:
   - Nodes can have child nodes, forming a hierarchy or tree-like structure.
   - Each level is a linked list, and child nodes can themselves be heads of another linked list.
2. **Two Pointers per Node**:
   - Each node has at least two pointers:
     - `next`: Points to the next node in the same level.
     - `child` (or `down`): Points to the head of a linked list in the next level.
3. **Multiple Levels**:
   - The list can have multiple levels, with each level being a standard linked list.
   - The depth of the hierarchy can vary depending on the application.
4. **Traversal Complexity**:
   - Traversal can be more complex than in a standard linked list because you need to navigate both horizontally (within a level) and vertically (across levels).
   - Common traversal methods include depth-first or breadth-first approaches.
5. **Applications**:
   - Used in scenarios where data has a nested or hierarchical structure, such as:
     - Representing a file system (directories and files).
     - Sparse matrices (where only non-zero elements are stored).
     - Nested data structures like JSON or XML.

Figure 4.2



6. **Memory Efficiency**:
   - Can be more memory-efficient for sparse data structures compared to arrays or matrices, as it only allocates memory for existing elements.
7. **Dynamic Structure**:
   - Like standard linked lists, multilevel linked lists are dynamic and can grow or shrink as needed.
8. **Complexity of Operations**:
   - Insertion, deletion, and search operations can be more complex due to the hierarchical nature of the list.
   - Time complexity depends on the depth and breadth of the structure.

Example Structure:

Copy

```
Level 1: 1 -> 2 -> 3 -> 4
         |    |
         |    5 -> 6
         7 -> 8
              |
              9 -> 10
```

- Node `1` has a child pointing to `7` (Level 2).
- Node `2` has a child pointing to `5` (Level 2).
- Node `8` has a child pointing to `9` (Level 3).

Common Operations:

1. **Traversal**:
   - Depth-first traversal: Explore all child nodes before moving to the next node in the same level.
   - Breadth-first traversal: Explore all nodes in the current level before moving to the next level.
2. **Flattening**:
   - Convert the multilevel linked list into a single-level linked list by merging all levels.
3. **Insertion/Deletion**:
   - Insert or delete nodes at specific levels or positions, ensuring the hierarchy is maintained.

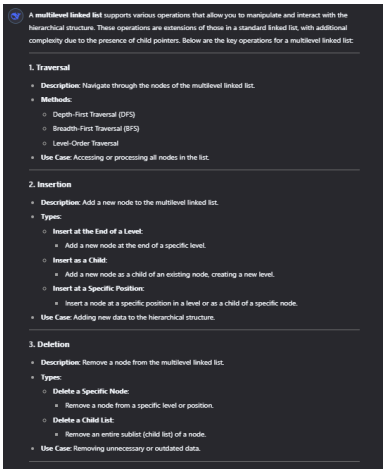
Use Cases:

- Representing hierarchical data (e.g., organizational charts, file systems).
- Storing sparse data efficiently.
- Implementing nested data structures in memory.

In summary, a multilevel linked list is a versatile data structure that extends the concept of a standard linked list to handle hierarchical or nested data efficiently.

Figure 4.3

Question 2: What are the different operations for multi-level linked lists?



A **multilevel linked list** supports various operations that allow you to manipulate and interact with the hierarchical structure. These operations are extensions of those in a standard linked list, with additional complexity due to the presence of child pointers. Below are the key operations for a multilevel linked list:

1. Traversal

- **Description**: Navigate through the nodes of the multilevel linked list.
- **Methods**:
  - Depth-First Traversal (DFS)
  - Breadth-First Traversal (BFS)
  - Level-Order Traversal
- **Use Case**: Accessing or processing all nodes in the list.

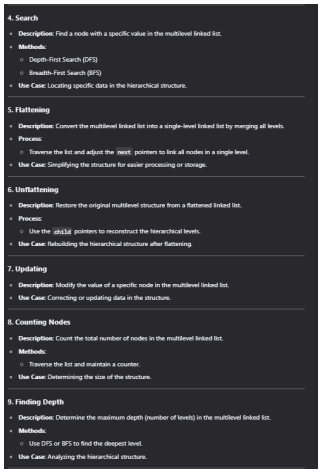
2. Insertion

- **Description**: Add a new node to the multilevel linked list.
- **Types**:
  - **Insert at the End of a Level**:
    - Add a new node at the end of a specific level.
  - **Insert as a Child**:
    - Add a new node as a child of an existing node, creating a new level.
  - **Insert at a Specific Position**:
    - Insert a node at a specific position in a level or as a child of a specific node.
- **Use Case**: Adding new data to the hierarchical structure.

3. Deletion

- **Description**: Remove a node from the multilevel linked list.
- **Types**:
  - **Delete a Specific Node**:
    - Remove a node from a specific level or position.
  - **Delete a Child List**:
    - Remove an entire sublist (child list) of a node.
- **Use Case**: Removing unnecessary or outdated data.

Figure 4.4



4. Search

- **Description**: Find a node with a specific value in the multilevel linked list.
- **Methods**:
  - Depth-First Search (DFS)
  - Breadth-First Search (BFS)
- **Use Case**: Locating specific data in the hierarchical structure.

5. Flattening

- **Description**: Convert the multilevel linked list into a single-level linked list by merging all levels.
- **Process**:
  - Traverse the list and adjust the `next` pointers to link all nodes in a single level.
- **Use Case**: Simplifying the structure for easier processing or storage.

6. Unflattening

- **Description**: Restore the original multilevel structure from a flattened linked list.
- **Process**:
  - Use the `child` pointers to reconstruct the hierarchical levels.
- **Use Case**: Rebuilding the hierarchical structure after flattening.

7. Updating

- **Description**: Modify the value of a specific node in the multilevel linked list.
- **Use Case**: Correcting or updating data in the structure.

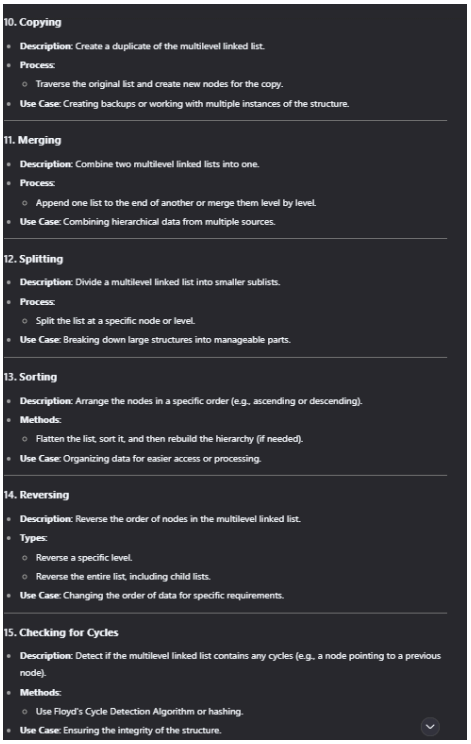
8. Counting Nodes

- **Description**: Count the total number of nodes in the multilevel linked list.
- **Methods**:
  - Traverse the list and maintain a counter.
- **Use Case**: Determining the size of the structure.

9. Finding Depth

- **Description**: Determine the maximum depth (number of levels) in the multilevel linked list.
- **Methods**:
  - Use DFS or BFS to find the deepest level.
- **Use Case**: Analyzing the hierarchical structure.

Figure 4.5



10. Copying

- **Description**: Create a duplicate of the multilevel linked list.
- **Process**:
  - Traverse the original list and create new nodes for the copy.
- **Use Case**: Creating backups or working with multiple instances of the structure.

11. Merging

- **Description**: Combine two multilevel linked lists into one.
- **Process**:
  - Append one list to the end of another or merge them level by level.
- **Use Case**: Combining hierarchical data from multiple sources.

12. Splitting

- **Description**: Divide a multilevel linked list into smaller sublists.
- **Process**:
  - Split the list at a specific node or level.
- **Use Case**: Breaking down large structures into manageable parts.

13. Sorting

- **Description**: Arrange the nodes in a specific order (e.g., ascending or descending).
- **Methods**:
  - Flatten the list, sort it, and then rebuild the hierarchy (if needed).
- **Use Case**: Organizing data for easier access or processing.

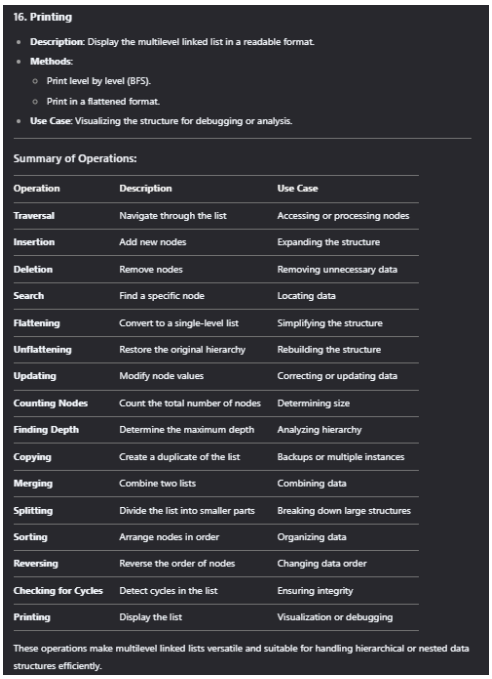
14. Reversing

- **Description**: Reverse the order of nodes in the multilevel linked list.
- **Types**:
  - Reverse a specific level.
  - Reverse the entire list, including child lists.
- **Use Case**: Changing the order of data for specific requirements.

15. Checking for Cycles

- **Description**: Detect if the multilevel linked list contains any cycles (e.g., a node pointing to a previous node).
- **Methods**:
  - Use Floyd's Cycle Detection Algorithm or hashing.
- **Use Case**: Ensuring the integrity of the structure.

Figure 4.6



16. Printing

- **Description**: Display the multilevel linked list in a readable format.
- **Methods**:
  - Print level by level (BFS).
  - Print in a flattened format.
- **Use Case**: Visualizing the structure for debugging or analysis.

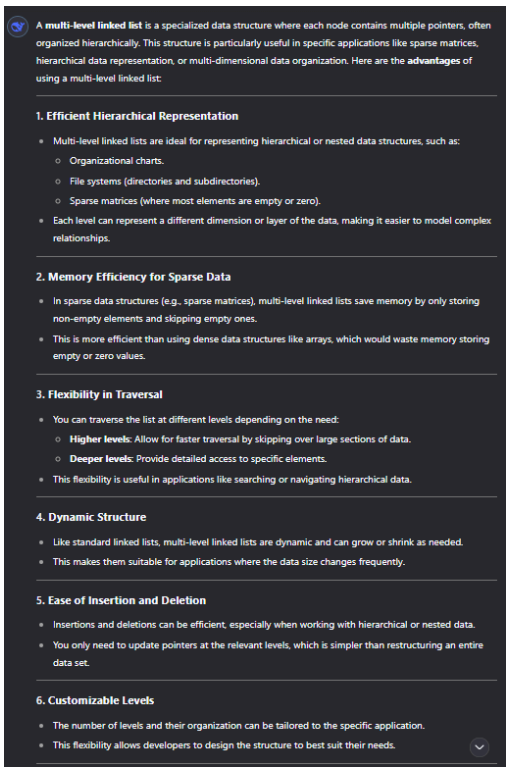
Summary of Operations:

Operation	Description	Use Case
Traversal	Navigate through the list	Accessing or processing nodes
Insertion	Add new nodes	Expanding the structure
Deletion	Remove nodes	Removing unnecessary data
Search	Find a specific node	Locating data
Flattening	Convert to a single-level list	Simplifying the structure
Unflattening	Restore the original hierarchy	Rebuilding the structure
Updating	Modify node values	Correcting or updating data
Counting Nodes	Count the total number of nodes	Determining size
Finding Depth	Determine the maximum depth	Analyzing hierarchy
Copying	Create a duplicate of the list	Backups or multiple instances
Merging	Combine two lists	Combining data
Splitting	Divide the list into smaller parts	Breaking down large structures
Sorting	Arrange nodes in order	Organizing data
Reversing	Reverse the order of nodes	Changing data order
Checking for Cycles	Detect cycles in the list	Ensuring integrity
Printing	Display the list	Visualization or debugging

These operations make multilevel linked lists versatile and suitable for handling hierarchical or nested data structures efficiently.

Figure 4.7

Question 3: What are the advantages of a multi-level linked list?



A **multi-level linked list** is a specialized data structure where each node contains multiple pointers, often organized hierarchically. This structure is particularly useful in specific applications like sparse matrices, hierarchical data representation, or multi-dimensional data organization. Here are the **advantages** of using a multi-level linked list:

1. Efficient Hierarchical Representation

- Multi-level linked lists are ideal for representing hierarchical or nested data structures, such as:
  - Organizational charts.
  - File systems (directories and subdirectories).
  - Sparse matrices (where most elements are empty or zero).
- Each level can represent a different dimension or layer of the data, making it easier to model complex relationships.

2. Memory Efficiency for Sparse Data

- In sparse data structures (e.g., sparse matrices), multi-level linked lists save memory by only storing non-empty elements and skipping empty ones.
- This is more efficient than using dense data structures like arrays, which would waste memory storing empty or zero values.

3. Flexibility in Traversal

- You can traverse the list at different levels depending on the need:
  - **Higher levels**: Allow for faster traversal by skipping over large sections of data.
  - **Deeper levels**: Provide detailed access to specific elements.
- This flexibility is useful in applications like searching or navigating hierarchical data.

4. Dynamic Structure

- Like standard linked lists, multi-level linked lists are dynamic and can grow or shrink as needed.
- This makes them suitable for applications where the data size changes frequently.

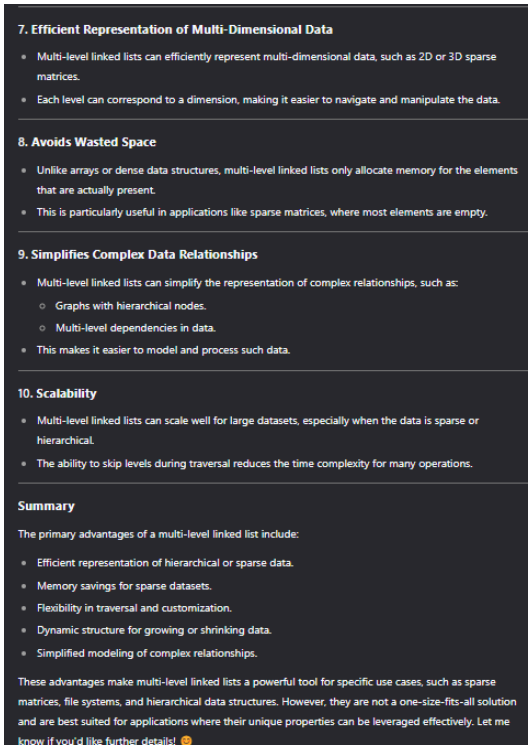
5. Ease of Insertion and Deletion

- Insertions and deletions can be efficient, especially when working with hierarchical or nested data.
- You only need to update pointers at the relevant levels, which is simpler than restructuring an entire data set.

6. Customizable Levels

- The number of levels and their organization can be tailored to the specific application.
- This flexibility allows developers to design the structure to best suit their needs.

Figure 4.8



7. Efficient Representation of Multi-Dimensional Data

- Multi-level linked lists can efficiently represent multi-dimensional data, such as 2D or 3D sparse matrices.
- Each level can correspond to a dimension, making it easier to navigate and manipulate the data.

8. Avoids Wasted Space

- Unlike arrays or dense data structures, multi-level linked lists only allocate memory for the elements that are actually present.
- This is particularly useful in applications like sparse matrices, where most elements are empty.

9. Simplifies Complex Data Relationships

- Multi-level linked lists can simplify the representation of complex relationships, such as:
  - Graphs with hierarchical nodes.
  - Multi-level dependencies in data.
- This makes it easier to model and process such data.

10. Scalability

- Multi-level linked lists can scale well for large datasets, especially when the data is sparse or hierarchical.
- The ability to skip levels during traversal reduces the time complexity for many operations.

Summary

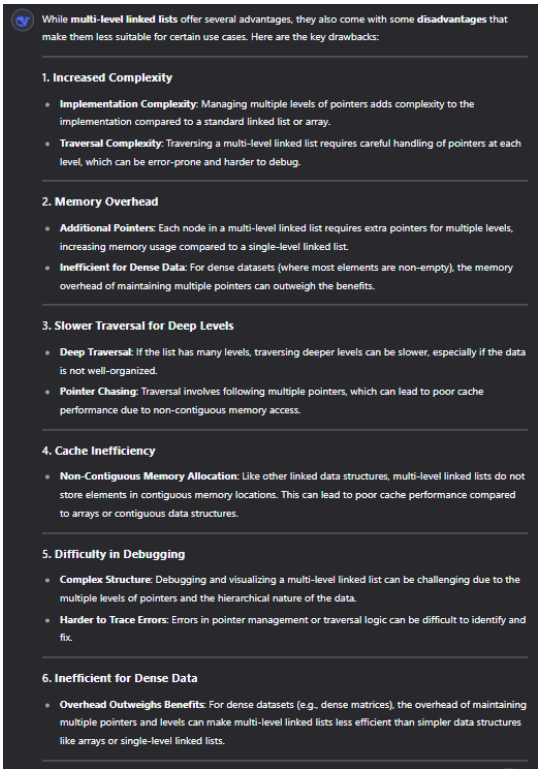
The primary advantages of a multi-level linked list include:

- Efficient representation of hierarchical or sparse data.
- Memory savings for sparse datasets.
- Flexibility in traversal and customization.
- Dynamic structure for growing or shrinking data.
- Simplified modeling of complex relationships.

These advantages make multi-level linked lists a powerful tool for specific use cases, such as sparse matrices, file systems, and hierarchical data structures. However, they are not a one-size-fits-all solution and are best suited for applications where their unique properties can be leveraged effectively. Let me know if you'd like further details!

Figure 4.9

Question 4: What are the disadvantages of a multi-level linked list?



While **multi-level linked lists** offer several advantages, they also come with some **disadvantages** that make them less suitable for certain use cases. Here are the key drawbacks:

1. Increased Complexity

- **Implementation Complexity**: Managing multiple levels of pointers adds complexity to the implementation compared to a standard linked list or array.
- **Traversal Complexity**: Traversing a multi-level linked list requires careful handling of pointers at each level, which can be error-prone and harder to debug.

2. Memory Overhead

- **Additional Pointers**: Each node in a multi-level linked list requires extra pointers for multiple levels, increasing memory usage compared to a single-level linked list.
- **Inefficient for Dense Data**: For dense datasets (where most elements are non-empty), the memory overhead of maintaining multiple pointers can outweigh the benefits.

3. Slower Traversal for Deep Levels

- **Deep Traversal**: If the list has many levels, traversing deeper levels can be slower, especially if the data is not well-organized.
- **Pointer Chasing**: Traversal involves following multiple pointers, which can lead to poor cache performance due to non-contiguous memory access.

4. Cache Inefficiency

- **Non-Contiguous Memory Allocation**: Like other linked data structures, multi-level linked lists do not store elements in contiguous memory locations. This can lead to poor cache performance compared to arrays or contiguous data structures.

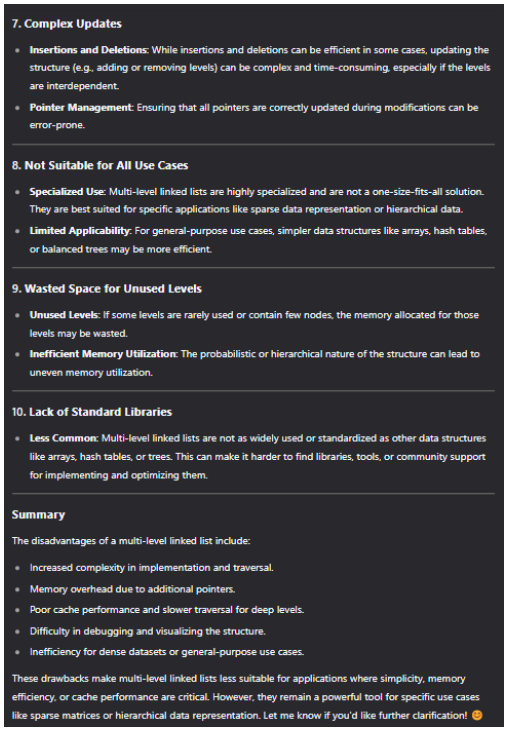
5. Difficulty in Debugging

- **Complex Structure**: Debugging and visualizing a multi-level linked list can be challenging due to the multiple levels of pointers and the hierarchical nature of the data.
- **Harder to Trace Errors**: Errors in pointer management or traversal logic can be difficult to identify and fix.

6. Inefficient for Dense Data

- **Overhead Outweighs Benefits**: For dense datasets (e.g., dense matrices), the overhead of maintaining multiple pointers and levels can make multi-level linked lists less efficient than simpler data structures like arrays or single-level linked lists.

Figure 4.10



7. Complex Updates

- **Insertions and Deletions**: While insertions and deletions can be efficient in some cases, updating the structure (e.g., adding or removing levels) can be complex and time-consuming, especially if the levels are interdependent.
- **Pointer Management**: Ensuring that all pointers are correctly updated during modifications can be error-prone.

8. Not Suitable for All Use Cases

- **Specialized Use**: Multi-level linked lists are highly specialized and are not a one-size-fits-all solution. They are best suited for specific applications like sparse data representation or hierarchical data.
- **Limited Applicability**: For general-purpose use cases, simpler data structures like arrays, hash tables, or balanced trees may be more efficient.

9. Wasted Space for Unused Levels

- **Unused Levels**: If some levels are rarely used or contain few nodes, the memory allocated for those levels may be wasted.
- **Inefficient Memory Utilization**: The probabilistic or hierarchical nature of the structure can lead to uneven memory utilization.

10. Lack of Standard Libraries

- **Less Common**: Multi-level linked lists are not as widely used or standardized as other data structures like arrays, hash tables, or trees. This can make it harder to find libraries, tools, or community support for implementing and optimizing them.

Summary

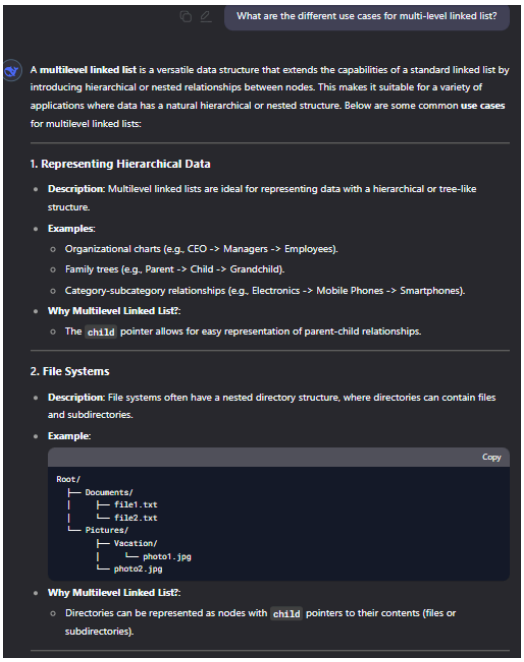
The disadvantages of a multi-level linked list include:

- Increased complexity in implementation and traversal.
- Memory overhead due to additional pointers.
- Poor cache performance and slower traversal for deep levels.
- Difficulty in debugging and visualizing the structure.
- Inefficiency for dense datasets or general-purpose use cases.

These drawbacks make multi-level linked lists less suitable for applications where simplicity, memory efficiency, or cache performance are critical. However, they remain a powerful tool for specific use cases like sparse matrices or hierarchical data representation. Let me know if you'd like further clarification!

Figure 4.11

Question 5: What are the different use cases for multi-level linked lists?



What are the different use cases for multi-level linked list?

A **multilevel linked list** is a versatile data structure that extends the capabilities of a standard linked list by introducing hierarchical or nested relationships between nodes. This makes it suitable for a variety of applications where data has a natural hierarchical or nested structure. Below are some common **use cases** for multilevel linked lists:

1. Representing Hierarchical Data

- **Description**: Multilevel linked lists are ideal for representing data with a hierarchical or tree-like structure.
- **Examples**:
  - Organizational charts (e.g., CEO -> Managers -> Employees).
  - Family trees (e.g., Parent -> Child -> Grandchild).
  - Category-subcategory relationships (e.g., Electronics -> Mobile Phones -> Smartphones).
- **Why Multilevel Linked List?**:
  - The `child` pointer allows for easy representation of parent-child relationships.

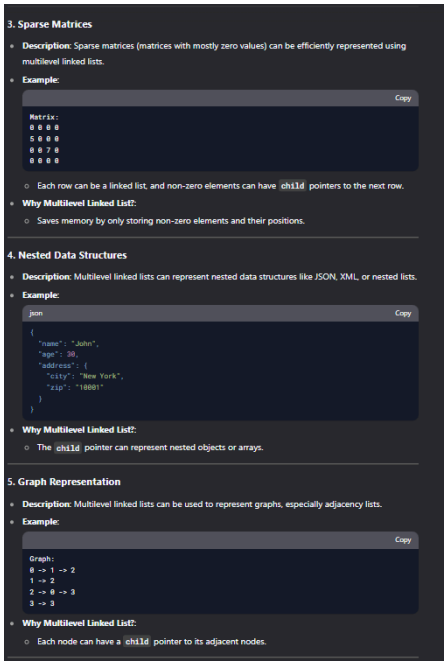
2. File Systems

- **Description**: File systems often have a nested directory structure, where directories can contain files and subdirectories.
- **Example**:

```
Root/
  ├── Documents/
  │     ├── file1.txt
  │     └── file2.txt
  └── Pictures/
        ├── Vacation/
        │     └── photo1.jpg
        └── photo2.jpg
```

- **Why Multilevel Linked List?**:
  - Directories can be represented as nodes with `child` pointers to their contents (files or subdirectories).

Figure 4.12



3. Sparse Matrices

- **Description**: Sparse matrices (matrices with mostly zero values) can be efficiently represented using multilevel linked lists.
- **Example**:

```
Matrix:
0 0 0 0
5 0 0 0
0 0 7 0
0 0 0 0
```

  - Each row can be a linked list, and non-zero elements can have `child` pointers to the next row.
- **Why Multilevel Linked List?**:
  - Saves memory by only storing non-zero elements and their positions.

4. Nested Data Structures

- **Description**: Multilevel linked lists can represent nested data structures like JSON, XML, or nested lists.
- **Example**:

```json
{
  "name": "John",
  "age": 30,
  "address": {
    "city": "New York",
    "zip": "10001"
  }
}
```

- **Why Multilevel Linked List?**:
  - The `child` pointer can represent nested objects or arrays.

5. Graph Representation

- **Description**: Multilevel linked lists can be used to represent graphs, especially adjacency lists.
- **Example**:

```
Graph:
0 -> 1 -> 2
1 -> 2
2 -> 0 -> 3
3 -> 3
```

- **Why Multilevel Linked List?**:
  - Each node can have a `child` pointer to its adjacent nodes.

Figure 4.13

6. Nested Menus or Navigation Systems

- **Description:** Multilevel linked lists can represent nested menus or navigation systems in applications.
- **Example:**

```
Main Menu
  ├── File
  |     ├── New
  |     ├── Open
  |     └── Save
  └── Edit
        ├── Cut
        ├── Copy
        └── Paste
```

- **Why Multilevel Linked List?:**
  - The `child` pointer can represent submenus or nested options.

7. Representing Multi-Level Document Outlines

- **Description:** Documents with multi-level headings (e.g., chapters, sections, subsections) can be represented using multilevel linked lists.
- **Example:**

```
Chapter 1
  ├── Section 1.1
  |     └── Subsection 1.1.1
  └── Section 1.2
```

- **Why Multilevel Linked List?:**
  - The hierarchical structure of the document can be easily mapped to the list.

8. Game Development

- **Description:** Multilevel linked lists can be used to represent game objects with nested hierarchies, such as scenes, characters, and inventory systems.
- **Example:**

```
Scene
  ├── Player
  |     ├── Inventory
  |     |     ├── Sword
  |     |     └── Shield
  |     └── Stats
  └── Enemies
        ├── Enemy1
        └── Enemy2
```

- **Why Multilevel Linked List?:**
  - The `child` pointer can represent nested game objects or components.

Figure 4.14

9. Representing Nested Comments or Threads

- **Description:** Multilevel linked lists can represent nested comment threads, such as those found in forums or social media platforms.
- **Example:**

```
Comment 1
  ├── Reply 1.1
  |     └── Reply 1.1.1
  └── Reply 1.2
```

- **Why Multilevel Linked List?:**
  - The `child` pointer can represent replies to comments.

10. Representing Nested Tasks or Dependencies

- **Description:** Multilevel linked lists can represent tasks with subtasks or dependencies.
- **Example:**

```
Task 1
  ├── Subtask 1.1
  |     └── Subtask 1.1.1
  └── Subtask 1.2
```

- **Why Multilevel Linked List?:**
  - The `child` pointer can represent subtasks or dependent tasks.

11. Representing Nested Data in Databases

- **Description:** Multilevel linked lists can represent nested or hierarchical data in databases, such as nested queries or relationships.
- **Example:**

```
Table 1
  ├── Row 1
  |     ├── Column 1
  |     └── Column 2
  └── Row 2
```

- **Why Multilevel Linked List?:**
  - The `child` pointer can represent nested rows or columns.

Figure 4.15

12. Representing Nested Data in Compilers

- **Description:** Multilevel linked lists can represent nested structures in compilers, such as abstract syntax trees (ASTs) or symbol tables.
- **Example:**

```
Function
  ├── Parameters
  |     ├── Param1
  |     └── Param2
  └── Body
        ├── Statement1
        └── Statement2
```

- **Why Multilevel Linked List?:**
  - The `child` pointer can represent nested statements or expressions.

Summary of Use Cases:

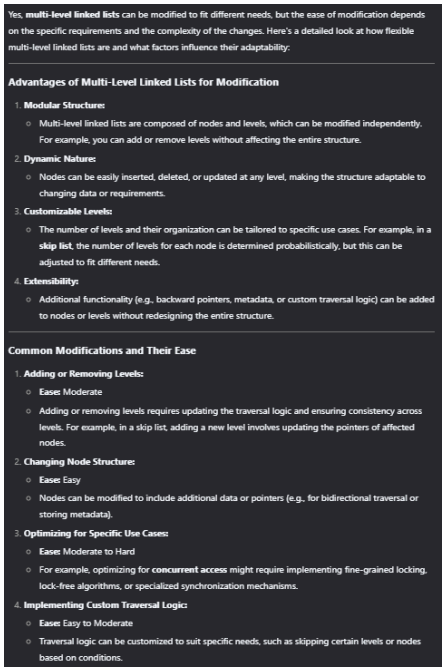
Use Case	Description
Hierarchical Data	Organizational charts, family trees, categories
File Systems	Directory and file structures
Sparse Matrices	Efficient storage of non-zero elements
Nested Data Structures	JSON, XML, nested lists
Graph Representation	Adjacency lists for graphs
Nested Menus	Multi-level navigation systems
Document Outlines	Chapters, sections, subsections
Game Development	Scenes, characters, inventory systems
Nested Comments	Forum or social media comment threads
Task Management	Tasks with subtasks or dependencies
Database Structures	Nested rows, columns, or queries
Compiler Structures	Abstract syntax trees, symbol tables

Why Use a Multilevel Linked List?

- **Hierarchy:** Naturally represents nested or hierarchical relationships.
- **Dynamic Structure:** Easily grows or shrinks as needed.
- **Memory Efficiency:** Only allocates memory for existing elements (useful for sparse data).
- **Versatility:** Suitable for a wide range of applications with nested or hierarchical data.

Figure 4.16

Question 6: Can a multi-level linked list be modified easily to fit different needs?



Yes, **multi-level linked lists** can be modified to fit different needs, but the ease of modification depends on the specific requirements and the complexity of the changes. Here's a detailed look at how flexible multi-level linked lists are and what factors influence their adaptability:

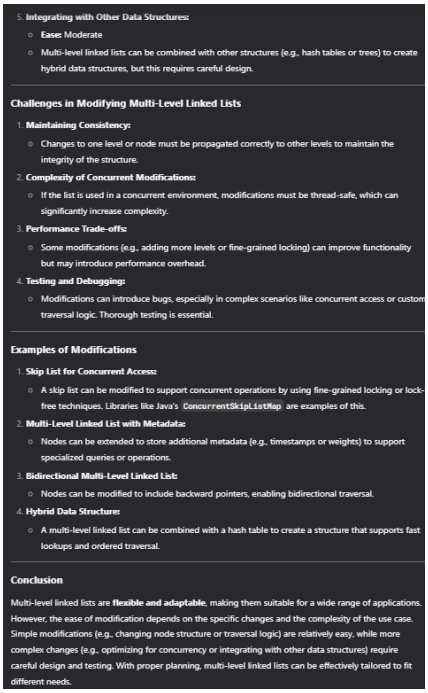
Advantages of Multi-Level Linked Lists for Modification

1. **Modular Structure:**
   - Multi-level linked lists are composed of nodes and levels, which can be modified independently. For example, you can add or remove levels without affecting the entire structure.
2. **Dynamic Nature:**
   - Nodes can be easily inserted, deleted, or updated at any level, making the structure adaptable to changing data or requirements.
3. **Customizable Levels:**
   - The number of levels and their organization can be tailored to specific use cases. For example, in a **skip list**, the number of levels for each node is determined probabilistically, but this can be adjusted to fit different needs.
4. **Extensibility:**
   - Additional functionality (e.g., backward pointers, metadata, or custom traversal logic) can be added to nodes or levels without redesigning the entire structure.

Common Modifications and Their Ease

1. **Adding or Removing Levels:**
   - **Ease:** Moderate
   - Adding or removing levels requires updating the traversal logic and ensuring consistency across levels. For example, in a skip list, adding a new level involves updating the pointers of affected nodes.
2. **Changing Node Structure:**
   - **Ease:** Easy
   - Nodes can be modified to include additional data or pointers (e.g., for bidirectional traversal or storing metadata).
3. **Optimizing for Specific Use Cases:**
   - **Ease:** Moderate to Hard
   - For example, optimizing for **concurrent access** might require implementing fine-grained locking, lock-free algorithms, or specialized synchronization mechanisms.
4. **Implementing Custom Traversal Logic:**
   - **Ease:** Easy to Moderate
   - Traversal logic can be customized to suit specific needs, such as skipping certain levels or nodes based on conditions.

Figure 4.17



5. **Integrating with Other Data Structures:**
   - **Ease:** Moderate
   - Multi-level linked lists can be combined with other structures (e.g., hash tables or trees) to create hybrid data structures, but this requires careful design.

Challenges in Modifying Multi-Level Linked Lists

1. **Maintaining Consistency:**
   - Changes to one level or node must be propagated correctly to other levels to maintain the integrity of the structure.
2. **Complexity of Concurrent Modifications:**
   - If the list is used in a concurrent environment, modifications must be thread-safe, which can significantly increase complexity.
3. **Performance Trade-offs:**
   - Some modifications (e.g., adding more levels or fine-grained locking) can improve functionality but may introduce performance overhead.
4. **Testing and Debugging:**
   - Modifications can introduce bugs, especially in complex scenarios like concurrent access or custom traversal logic. Thorough testing is essential.

Examples of Modifications

1. **Skip List for Concurrent Access:**
   - A skip list can be modified to support concurrent operations by using fine-grained locking or lock-free techniques. Libraries like Java's `ConcurrentSkipListMap` are examples of this.
2. **Multi-Level Linked List with Metadata:**
   - Nodes can be extended to store additional metadata (e.g., timestamps or weights) to support specialized queries or operations.
3. **Bidirectional Multi-Level Linked List:**
   - Nodes can be modified to include backward pointers, enabling bidirectional traversal.
4. **Hybrid Data Structure:**
   - A multi-level linked list can be combined with a hash table to create a structure that supports fast lookups and ordered traversal.

Conclusion

Multi-level linked lists are **flexible and adaptable**, making them suitable for a wide range of applications. However, the ease of modification depends on the specific changes and the complexity of the use case. Simple modifications (e.g., changing node structure or traversal logic) are relatively easy, while more complex changes (e.g., optimizing for concurrency or integrating with other data structures) require careful design and testing. With proper planning, multi-level linked lists can be effectively tailored to fit different needs.

Figure 4.18

Evaluating the response of DeepSeek:

Question 1: What are the properties of a multi-level linked list?

Accuracy	The chatbot is **accurate** in its description of the hierarchy structure, operations and the complexity of operations.
	
	Hierarchy Structure
	- The response stated by the chatbot **aligns** with the explanation by Geeksforgeeks (2022), which describes the structure and properties of a multi-level linked list.
	
	Operations
	- The chatbot response **correctly** identifies the following operations:
	  - Traversal
	  - Geeksforgeeks (2024) showed what are the properties and differences between Depth-first traversal (DFS) and Breadth-first traversal (BFS)
	  - Flattening
	  - Geeksforgeeks (2024) showed how to flatten a multi-level linked list using level-order traversal to a single list.

	- Insertion/Deletion - HeyCoach(2024) mentioned that insertion or deletion is part of the common operations of linked list
Completeness	The chatbot missed out on providing a more comprehensive list of the different common operations, such as updating and searching for a node of the multi-level linked list. In addition, while the chatbot mentioned the complexity of operations, it did not give the exact time complexity for each operation, but directly stated that the operations would be more complex. Thus, there are no examples illustrating the different time complexity of the operations, making it hard to understand the impact of the time complexity of the operations.
Clarity	The response of the chatbot is clear and straightforward. The chatbot also provided a diagram of the structure of a multi-level linked list, allowing it to be easier to follow and understand.
Relevance	The content given by the chatbot is somewhat relevant to the question. Even though it directly addresses the multi-level linked list.

Question 2: What are the operations for the multi-level linked list?

Accuracy	The response provided an accurate summary of the different operations of a multi-level linked list, which includes: - Traversal - Insertion - Deletion - Search - Flattening - Counting - Updating All of these operations were mentioned by HeyCoach(2024) and Geeksfogeeks(2024). Additionally, other operations such as merging, splitting, sorting, reversing and checking for cycles can be referenced to how the standard linked list handles it.

Completeness	The response given is complete, and it mentioned many different operations, such as merging, splitting and sorting. Some of these operations are less commonly mentioned for a multi-level linked list.
Clarity	The clarity of response is clear and straightforward, as the chatbot explains how the different operations work and its use cases. This greatly increases its readability, making it easy to follow.
Relevance	The details given are relevant to the topic and even go beyond the basic operations, such as merging, splitting, sorting, and reversing, showing the many possibilities that can be done with the data structure.

Question 3: What are the advantages of a multi-level linked list?

Accuracy	The response given is accurate, as the advantages provided were highlighted by SparkCodeHub(2022) and HeyCoach(2024). They pointed out the benefits such as its flexibility and dynamic structure, which aligned with the points provided by the chatbot.
Completeness	The response provided is completed, with it explaining further why each point is advantageous. However, it can be further improved by providing how the insertion and deletion of a multi-level linked list are efficient by showing its time complexity.
Clarity	The clarity of response is clear, giving bullet points and headings to organise the different points, and even summarising at the end. In addition, the points are straight to the point, allowing it to be easier to understand.
Relevance	The content provided is highly relevant to the question, where it directly addresses the different advantages of the data structure.

Question 4: What are the disadvantages of a multi-level linked list?

Accuracy	The disadvantages listed correctly highlight the different challenges of multi-linked lists, such as memory overhead, complex updates and not being suitable for all use cases, with SparkCodeHub(2022) and HeyCoach(2024) supporting the different drawbacks mentioned.
Completeness	The response given is mostly completed, however, it would be good to mention what are the specific use case examples that will be unsuitable for this data structure and further explain why it is difficult

	to do complex updates.
Clarity	The clarity of response is clear, with bullet points and headings to organise the different points, and summarising at the end. In addition, the points are straight to the point, allowing it to be easier to understand.
Relevance	The content provided is highly relevant to the question, where it directly addresses the different disadvantages of the data structure.

Question 5: What are the different use cases for a multi-level linked list?

Accuracy	The chatbot response accurately shows that the various use cases mentioned, such as game development and file systems can be implemented, as supported by SparkCodeHub(2022) and HeyCoach(2024).
Completeness	The response given is completed, with it giving detailed descriptions of how the multi-level linked list works. However, not all of the use cases were found in the response, such as applying network systems (HeyCoach, 2024) and artificial intelligence and machine learning (SparkCodeHub, 2022)
Clarity	With detailed descriptions and even a diagram to show how the multi-level linked list will work in the use case, it is easy to understand and follow.
Relevance	The response given is highly relevant to the question, where it addresses the different use cases found and how to implement the multi-level linked list.

Question 6: Can a multi-level linked list be modified easily to fit different needs?

Accuracy	The response given is somewhat accurate in terms of its response to the question. The chatbot mentioned the advantages of the data structure for modifications, the different modifications and their ease, challenges and different examples of modifications. For the advantages of the data structure for modifications, the different modifications and their ease, and challenges, the chatbot accurately states and describes how the points relate. However, the examples of modifications, it is not very accurate. The

	first point states ‘Skip List for Concurrent Access’. Skip List is a different structure of the multi-level linked list, thus, it is not accurate as it gave an example for a different data structure.
Completeness	The chatbot gave a complete response to the question as it mentioned the different advantages of the structure and the common modifications and examples, allowing it to be easy to see how the different points relate to each other.
Clarity	The clarity of chatbot response is partially clear as even though it provides a short explanation of the different points, it does not go in-depth into how each feature can be implemented. For instance, on the examples of modifications, the chatbot should go more into detail on how the multi-level linked list can be modified, rather than just stating the different modifications of the multi-level linked list.
Relevance	The chatbot response is mostly relevant. For example, when the chatbot gave examples of modifications, it mentioned ‘skip list for Concurrent Access’, which is not relevant to the question as it states a different data structure for modification.

Summary:
Overall, the chatbot breaks down information and usually shows how the points work in visual representation, allowing it to be easier to understand and follow. Additionally, it will usually stay on topic of the question it was prompted, giving detailed information on the topic. However, there are times when the chatbot will slightly go off-topic, thus, it is important to read through carefully to make sure it aligns with the topic. Additionally, when asking the chatbot questions, it is beneficial to be specific about a particular aspect of the data structure to get a clearer explanation. Otherwise, the chatbot will only give a broad overview of multiple different points without going into deeper details.

Thus, the chatbot response could be improved by asking follow-up questions for a vague question, ensuring the response given will match the users' needs more precisely, giving a more in-depth response.

5) Implement Application

Overview Of Application

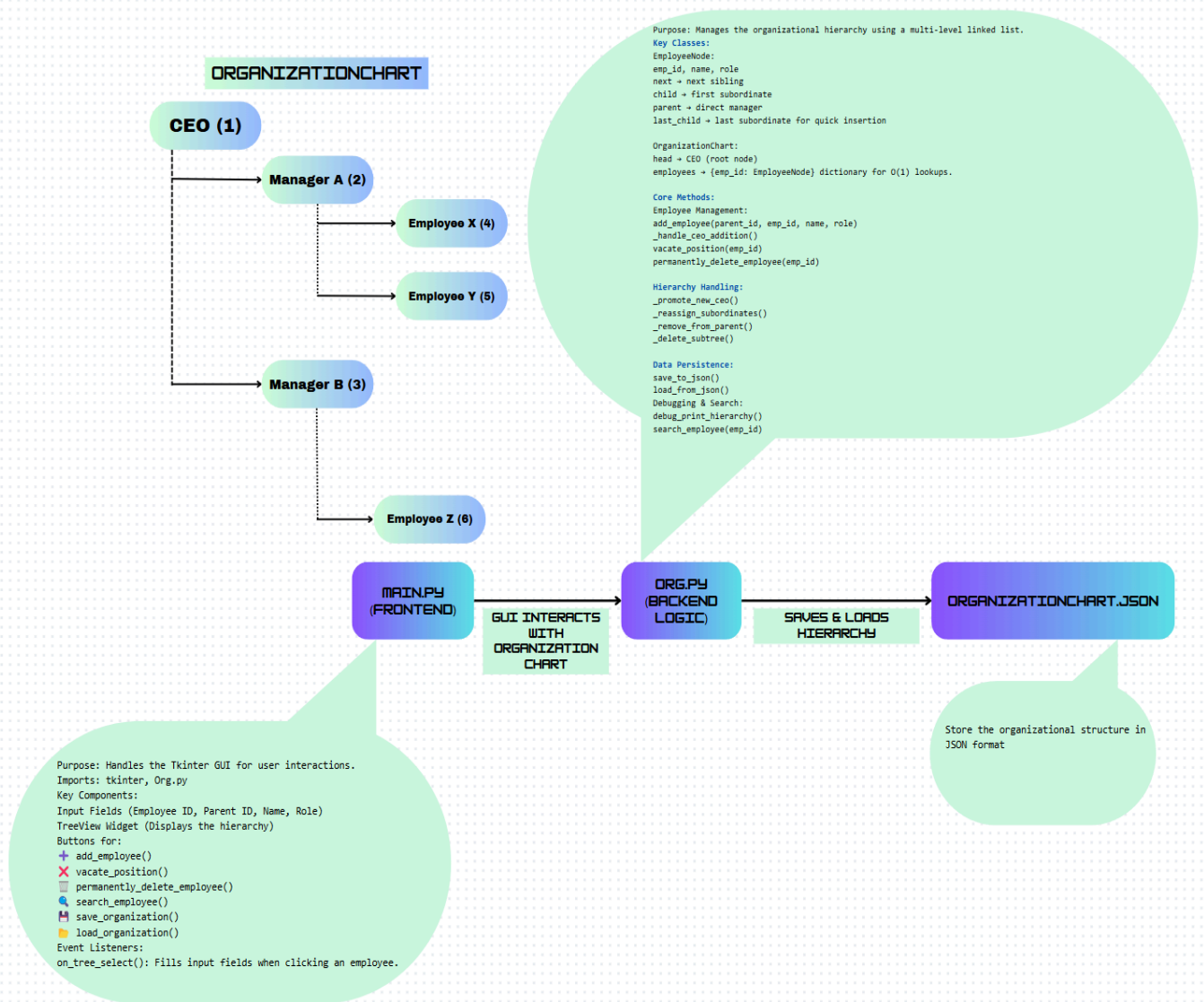


Figure 5.1

The application chosen was an Organization Chart Management System (Figure 5.1) that enables big companies to visualise and manage their company hierarchy. The application provides features such as adding employees, deleting positions, searching for employees, saving & loading organization data and vacating positions for new people to take over.

The hierarchical structure of the companies naturally fits a **multi-level linked list** hence it was chosen as the **main primary data structure**, where each employee node can have a

child (which we set as our subordinate) and a **next pointer (sibling at the same level)**. This structure efficiently represents the complex organisational relationships.

Reason for using **Multi-Level Linked List** for modelling the organisation:

- Each node represents an **employee** with attributes like their ID, Name & Roles.
- The **child pointer** represents a subordinate (Like Direct Reporting).
- The **next pointer** represents the employees that are on the same hierarchical level ("siblings").
- Multi-level linked lists are efficient in dealing with dynamic modifications such as inserting and removing employees removing the requirement of costly array shifts.
- It supports recursive traversal, enabling hierarchy-based operations (e.g. displaying of structure) fairly straight forward.

Functional Components

The implementation supports multiple operations including:

- Adding employees under the supervisor
- Vacating Positions without deleting hierarchy
- Permanently deleting employees and reassigning subordinates
- Searching for employees by ID (Using a dictionary)
- Saving and Loading the organization structure from a file

A dictionary (hash table) is used as a helper data structure to ensure for **fast lookup** of O(1) of employees via their unique ID.

1. Adding and updating employees.

   As organisations expand and hire more employees, they must update their hierarchy database to include the new hires and their relative positions. This feature allows for the adding of new employee roles as well as updating already existing roles within the hierarchy.

   To add a new role to the hierarchy, we need to create a new node. Each node is comprised of :
   - The employee's name, role and unique ID.
   - The employee's direct supervisor's ID (parent ID)
     - This does not apply to the CEO position as it is at the top of the hierarchy and thus has no parent.

How to add a new role:

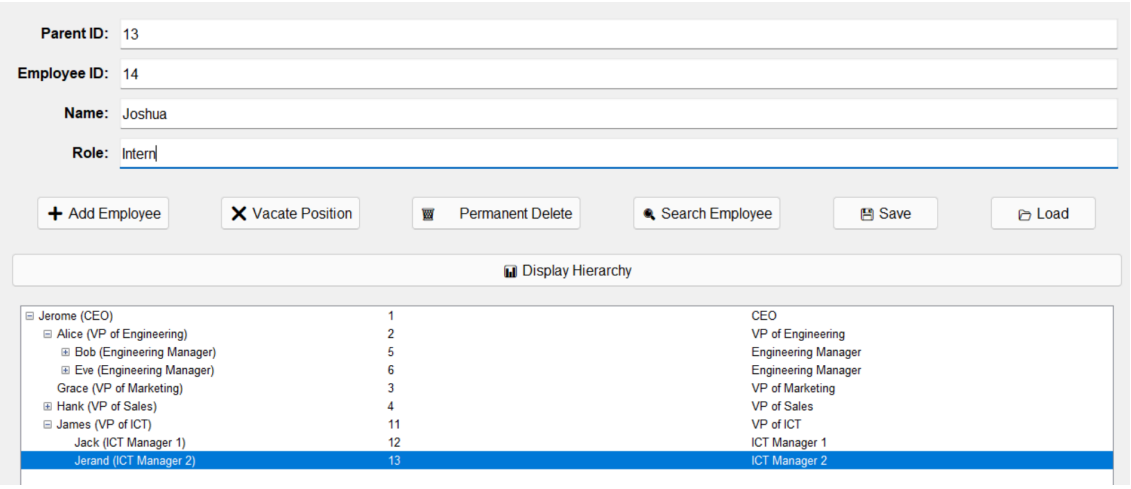


Figure 5.2 (example to add a new intern position under Jerand)

1. Enter the new employee's name, role and unique ID into the UI.
2. Enter the ID of their supervisor
    a. (In the Figure 5.2 example we use Jerand who has ID 13)
3. Click on the Add Employee button.
    a. A pop-up window should appear confirming the successful addition of the new employee.
4. You can either search the tree view or use the search employee to verify the new role has been added.d
5. Click on the Save button to save the new employee to the database.

How to update a vacant role:

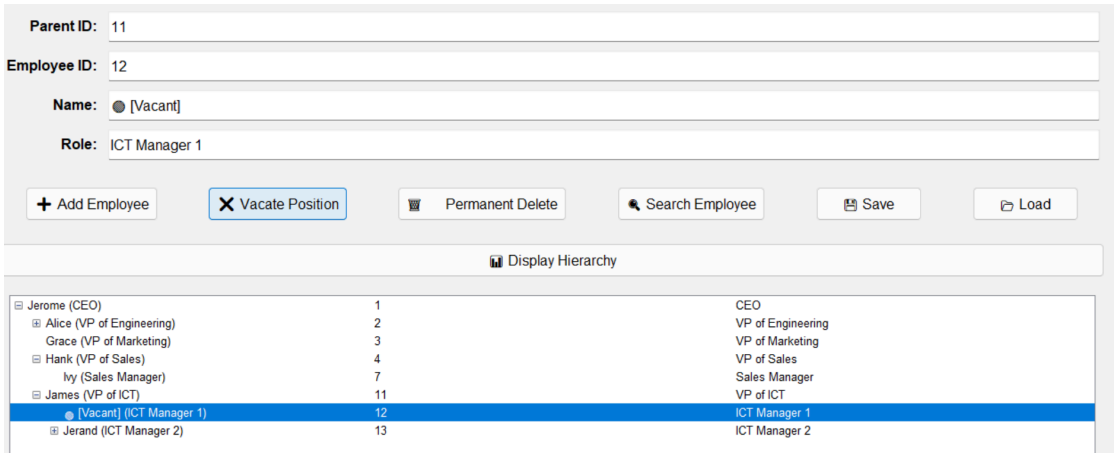


Figure 5.3

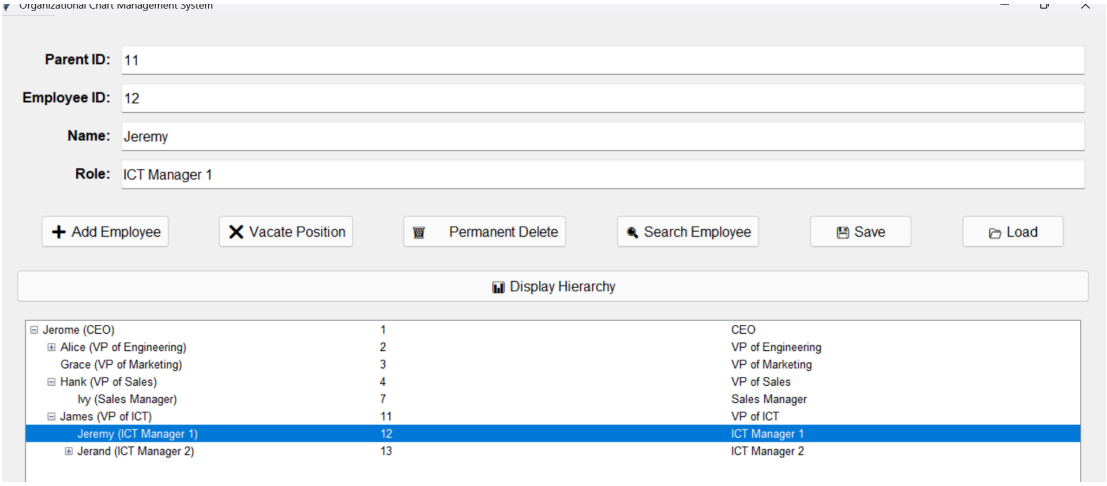


Figure 5.4

1. Find the vacant role you want to fill and select it.
    a. This should automatically fill in the parent and employee ID as well as the role (See Figure 5.4)
2. Replace the placeholder name with the new employee's name
3. Click on the Add Employee button.
    a. A pop-up window should appear confirming the successful updating of the role.
6. You can either search the tree view or use the search employee to verify the updated details of that role.
7. Click on the Save button to save the changes to the database.

2. Vacating Position

Positions in the organization may become vacant due to either the firing of an existing employee or a recent promotion. In these scenarios, we want to preserve the role while removing the previous employees' details allowing for easier replacements. This feature allows employee names to be removed from the hierarchy without deleting the node allowing a new employee to be assigned to that role using the Add Employee feature.

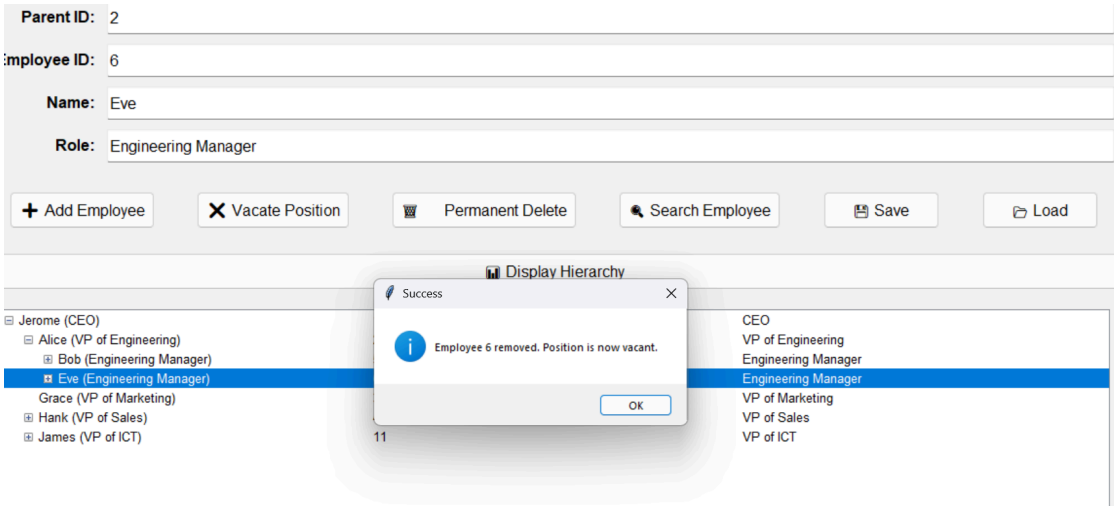


Figure 5.5

To vacate a role, select the role in the Treeview and click the Vacate Position button. A pop-up will indicate the successful vacating of that role.

3. Permanently Deleting an Employee

Sometimes due to downsizing or certain roles becoming obsolete, roles have to be deleted from the hierarchy. This feature allows roles to be deleted. When roles with subordinates are deleted, it allows the user to choose whether they want to also delete all the subordinates' positions or promote them to fill the void in the hierarchy.

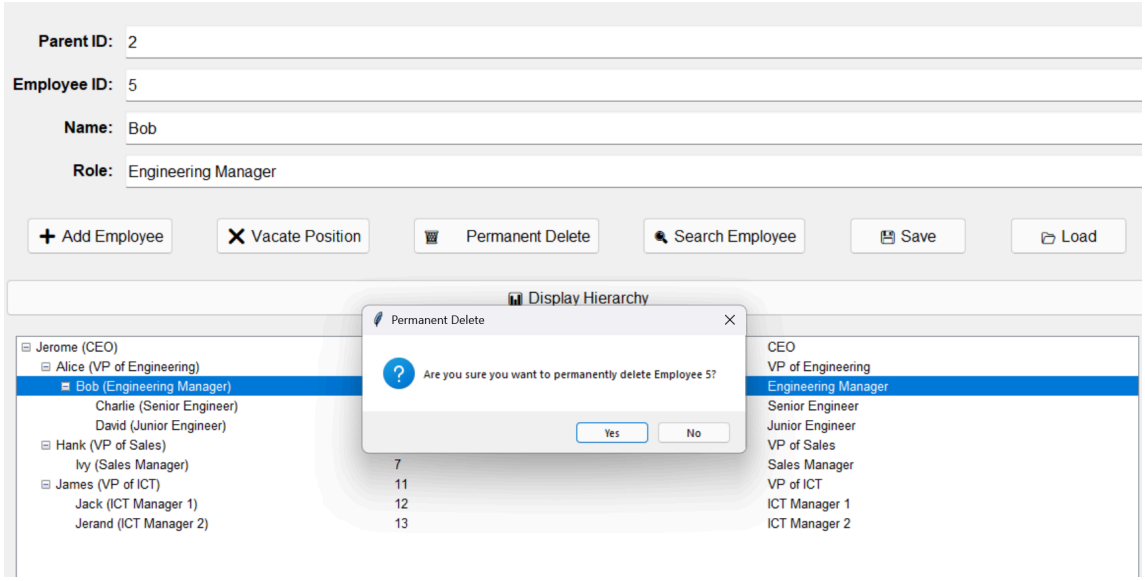


Figure 5.6

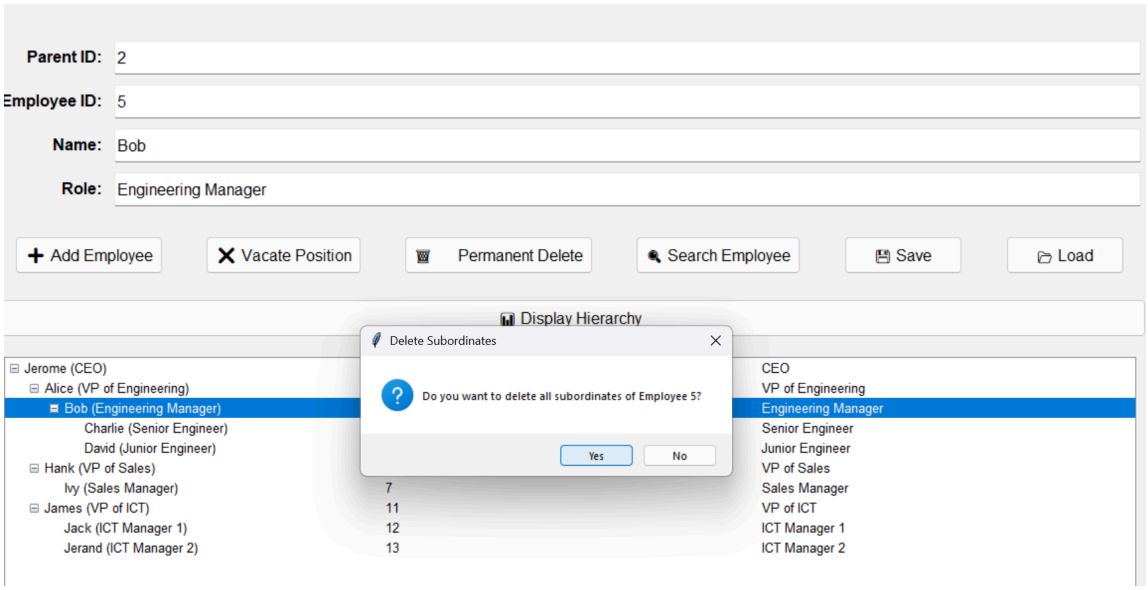


Figure 5.7

To delete a role, select the role on the treeview and click the Permanent Delete button. A pop-up will prompt you to confirm the deletion of that role as well as any subordinate roles. If you click on the deletion of the subordinate roles, they will be promoted one level up the hierarchy.

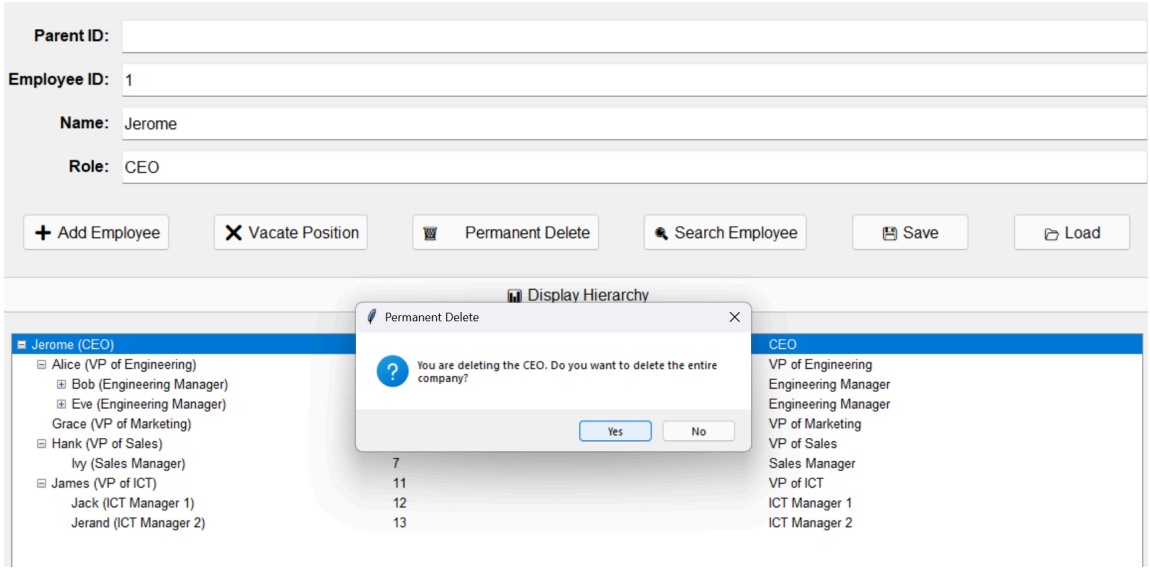


Figure 5.8

When deleting the CEO a different pop-up will be made after confirming asking if you want to retain or delete the rest of the company. If you click no, the first child node will move to populate the position.

4. Searching for an Employee

When there are many employees within a company, it becomes difficult to find the one you are looking for in the database. Having a search feature makes finding individual employee details more convenient. This feature allows a fast search of employee details using their Employee ID.

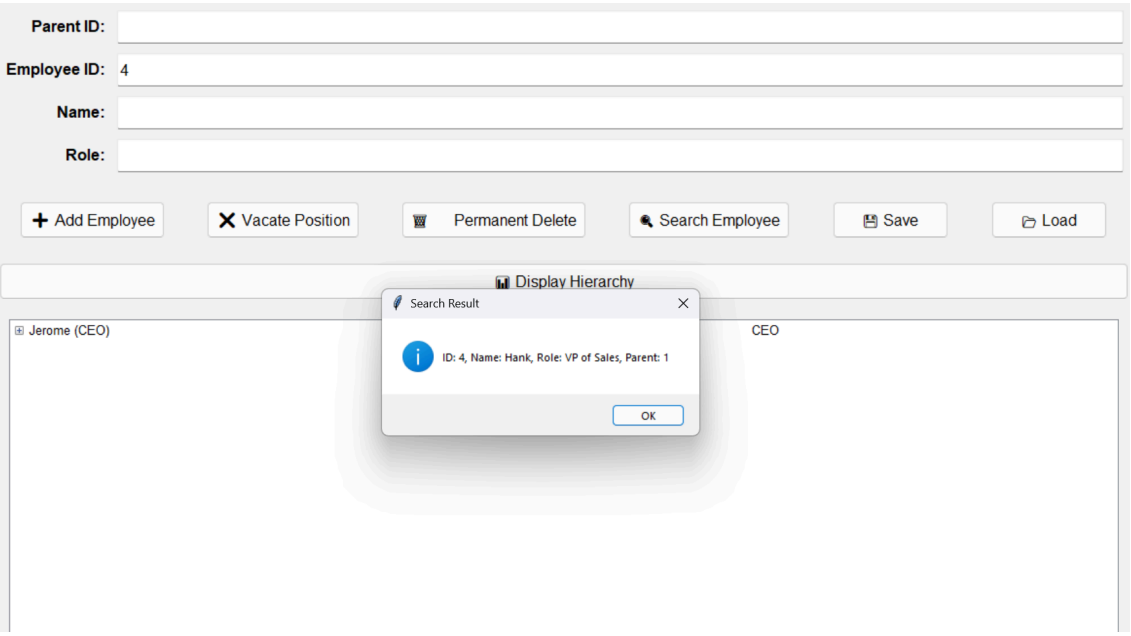


Figure 5.9

To search for an employee's details, enter their Employee ID and click the Search Employee button. A pop-up will appear containing the employee's details.

5. Saving and Loading Data

The load and save functions allow the program to read and write the database respectively. This allows other database snapshots to be loaded into the program and for changes made in the program to update the current database. The database is written in JSON format.

To load or save click their respective buttons. A pop-up will appear to confirm successful loading or saving.

Justification of Data Structures & Methods Chosen

Multi-Level Linked List Structure

```
class EmployeeNode:
    def __init__(self, emp_id, name, role):
        self.emp_id = emp_id
        self.name = name
        self.role = role
        self.next = None  # Pointer to next sibling
        self.child = None  # Pointer to first child
        self.parent = None  # Pointer to parent
        self.last_child = None  # Pointer to last child for fast insert
```

Figure 5.10

As mentioned, a **multi-level linked list** was chosen as the primary data structure due to it being able to model hierarchical structures like an organisation chart. A multi-level linked list has **flexible hierarchy management** (employees can be dynamically added, moved or removed), **efficient traversal** (using child and next pointers allowing for fast depth-first traversal) and **preserves structure** (unlike arrays or trees, this allows vacant positions without restructuring).

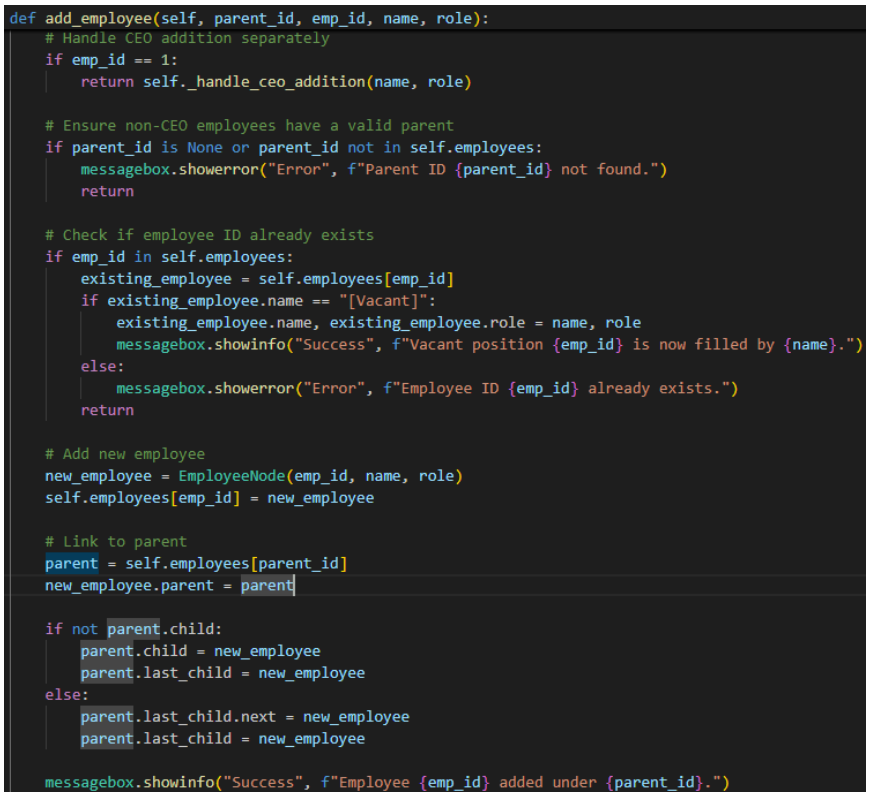
Helper Dictionary

```
class OrganizationChart:
    def __init__(self):
        self.head = None  # Root node (CEO)
        self.employees = {}  # Dictionary for O(1) lookup
```

Figure 5.11

The helper data structure dictionary provides O(1) access to employees using emp_id. Without this helper dictionary, searching through the hierarchy takes O(n) time.

Insertion (add_employee)



```
def add_employee(self, parent_id, emp_id, name, role):
    # Handle CEO addition separately
    if emp_id == 1:
        return self._handle_ceo_addition(name, role)

    # Ensure non-CEO employees have a valid parent
    if parent_id is None or parent_id not in self.employees:
        messagebox.showerror("Error", f"Parent ID {parent_id} not found.")
        return

    # Check if employee ID already exists
    if emp_id in self.employees:
        existing_employee = self.employees[emp_id]
        if existing_employee.name == "[Vacant]":
            existing_employee.name, existing_employee.role = name, role
            messagebox.showinfo("Success", f"Vacant position {emp_id} is now filled by {name}.")
        else:
            messagebox.showerror("Error", f"Employee ID {emp_id} already exists.")
        return

    # Add new employee
    new_employee = EmployeeNode(emp_id, name, role)
    self.employees[emp_id] = new_employee

    # Link to parent
    parent = self.employees[parent_id]
    new_employee.parent = parent

    if not parent.child:
        parent.child = new_employee
        parent.last_child = new_employee
    else:
        parent.last_child.next = new_employee
        parent.last_child = new_employee

    messagebox.showinfo("Success", f"Employee {emp_id} added under {parent_id}.")
```

Figure 5.12

Handles CEO addition separately, adds new roles or updates existing roles to the hierarchy. Complexity: O(1) by using the last child pointer to achieve consistent time when adding new child nodes.

Deletion (permanently_delete_employee)

```
def permanently_delete_employee(self, emp_id):
    """Permanently delete an employee and handle subordinates accordingly."""
    employee = self.employees.get(emp_id)
    if not employee:
        messagebox.showerror("Error", "Employee not found.")
        return

    if not messagebox.askyesno("Permanent Delete", f"Are you sure you want to permanently delete Employee {emp_id}?"):
        return

    # Handle CEO deletion separately
    if employee == self.head:
        self._handle_ceo_deletion(employee)
        return

    parent = employee.parent

    if employee.child:
        self._handle_subordinates_deletion(employee, parent)

    self._remove_from_parent(employee, parent)
    self._remove_employee(employee)

    messagebox.showinfo("Success", f"Employee {emp_id} has been permanently deleted.")
```

Figure 5.13

Handles CEO deletion separately, and can reassign or delete subordinates. Time Complexity: O(n) worse case if the whole subtree is removed.

Vacating a Position (vacate_position)

```
def vacate_position(self, emp_id):
    employee = self.employees.get(emp_id)
    if not employee:
        messagebox.showerror("Error", "Employee not found.")
        return
    employee.name = "[Vacant]"
    messagebox.showinfo("Success", f"Employee {emp_id} removed. Position is now vacant.")
```

Figure 5.14

Do not delete the employee, preserving the hierarchy.
Complexity: O(1) since only the field is updated.

Searching of Employee (search_employee)

```
def search_employee(self, emp_id):
    employee = self.employees.get(emp_id)
    if employee:
        parent_id = employee.parent.emp_id if employee.parent else "None"
        return f"ID: {employee.emp_id}, Name: {employee.name}, Role: {employee.role}, Parent: {parent_id}"
    return "Employee not found."
```

Figure 5.15

Complexity: O(1) using dictionary lookup

Usage of Depth-First Search (DFS)
[display_hierarchy() via using the _add_to_tree(), serialize_employee() and _deserialize_employee()]

display_hierarchy() via using the _add_to_tree()

```
def display_hierarchy(self):
    """Refreshes the TreeView hierarchy and ensures a valid CEO exists."""
    self.tree.delete(*self.tree.get_children())  # Clear tree

    if not self.org.head:
        print("No CEO found. Organization is empty.")
        messagebox.showinfo("Info", "The organization is now empty.")
        return  # Exit without displaying anything

    print(f"Displaying hierarchy from CEO: {self.org.head.name} ({self.org.head.emp_id})")
    self._add_to_tree("", self.org.head)

def _add_to_tree(self, parent, node):
    """Recursively adds employees to the tree view."""
    display_text = f"{node.name} ({node.role})"
    if node.name == "[Vacant]":
        display_text = f"🔴 [Vacant] ({node.role})"  # Highlight vacant positions

    # FIXED: Store both `ID` and `Role` properly while keeping hierarchy
    item = self.tree.insert(parent, "end", text=display_text, values=(node.emp_id, node.role))

    # Recursively add all children
    child = node.child
    while child:
        self._add_to_tree(item, child)
        child = child.next
```

Figure 5.16

How DFS is used:
_add_to_tree() recursively processes each node before visiting a **preorder DFS traversal** (process node -> visit children).
The function visits each **employee** once and recursively processes subordinates, making it a top to down traversal.

Complexity: O(n) where N is the number of employees. Each employee is visited once.

When using DFS, the recursive approach is intuitive for hierarchical data like organisation charts. It also allows for the direct parent-child relationships to be preserved in the tree structure.

serialize_employee()

```
def _serialize_employee(self, employee):
    """Converts an EmployeeNode into a dictionary format for JSON storage."""
    if not employee:
        return None

    data = {
        "emp_id": employee.emp_id,
        "name": employee.name,
        "role": employee.role,
        "last_child": employee.last_child.emp_id if employee.last_child else None,  # Restore last_child
        "subordinates": []
    }

    child = employee.child
    while child:
        data["subordinates"].append(self._serialize_employee(child))
        child = child.next

    return data
```

Figure 5.17

How DFS is used:

The function serialises the organisation before using **DFS traversal**. Each employee is processed before visiting and serialising their children. The structure is saved recursively while maintaining hierarchical order.

Time Complexity: O(N) where N is the number of employees. Each employee is visited once.

When using DFS, it allows for hierarchical data to be stored efficiently in a structured JSON format. The recursive approach mirrors the whole organisation structure, making it easy to restore.

_deserialize_employee()

```python
def _deserialize_employee(self, data, parent=None):
    """Reconstructs an EmployeeNode from JSON data while maintaining `last_child`."""
    if not data:
        return None

    emp = EmployeeNode(data["emp_id"], data["name"], data["role"])
    emp.parent = parent
    self.employees[emp.emp_id] = emp

    prev_child = None
    first_child = None

    for child_data in data.get("subordinates", []):
        child = self._deserialize_employee(child_data, emp)
        if prev_child:
            prev_child.next = child
        else:
            first_child = child
        prev_child = child

    emp.child = first_child
    emp.last_child = prev_child  # Restore last_child reference

    return emp
```

Figure 5.18

How DFS is used:

The function recursively restores the organisation chart using DFS traversal, each employee is created before processing subordinates to ensure hierarchical consistency by linking the subordinates correctly.

Time Complexity: O(N) where N is the number of employees. Each employee is created once.

When using DFS, it enables recursive reconstruction of the hierarchical structure ensuring that the parent-child relationship remains intact during deserializing.

Usage of AI for Code Generation

In developing this application, ChatGPT was utilised as an AI-assisted coding tool to generate, refine, and troubleshoot parts of the organisation chart implementation. The chatbot was useful in generating the base implementation of the data structures used, multi-linked list & helper dictionary, traversal algorithms, serialisation methods and even helping to set up and integrate Tkinter for Graphical User Interface (GUI).

AI Prompts Used

Main Prompt:
"I want to create a company hierarchy chart where users can insert, search, delete, vacate, load, and save the hierarchy using a JSON file. The primary data structure should be a multi-level linked list, with a dictionary for efficient searching. The hierarchy should be displayed in a GUI using Tkinter. This should be done in Python."

AI's Response:

1. Multi-Level Linked List Structure *(Look at Modifications to code)*
    - AI helped to generate the multi-linked list structure with a class called EmployeeNode.
    - However, it failed to have the correct pointers and did not include the helper dictionary.

2. Insertion and Searching *(Look at Modifications to code)*
    - AI provided a method that does not explicitly have CEO handling and overwrites the parents' children instead of linking. They also ignore the **last_child** optimization.

3. Deletion & Vacating *(Look at Modifications to code)*
    - AI provided a method that was pointing incorrectly in reassignments, it did not properly reconnect subordinates when a node was deleted, leading to orphaned nodes.
    - AI failed to create the vacate function & handling for the vacating function in insertion.

4. Saving & Loading the Hierarchy (JSON Serialisation) *(Look at Modifications to code)*
    - AI suggested a recursive approach for serialisation but did not maintain the last_child reference leading to an improper reconstruction.

5. Tkinter Integration
    - AI's initial attempt only generated a simple Treeview widget without interactive features like a button for adding, deleting, and searching employees, not even handling or linking UI to backend functions or even the ability to save and load the organisation chart. (As shown below)

AI Code before further prompting

```
import tkinter as tk
from tkinter import ttk

class OrgChartApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Company Hierarchy")

        self.tree = ttk.Treeview(root)
        self.tree.pack(fill=tk.BOTH, expand=True)

root = tk.Tk()
app = OrgChartApp(root)
root.mainloop()
```

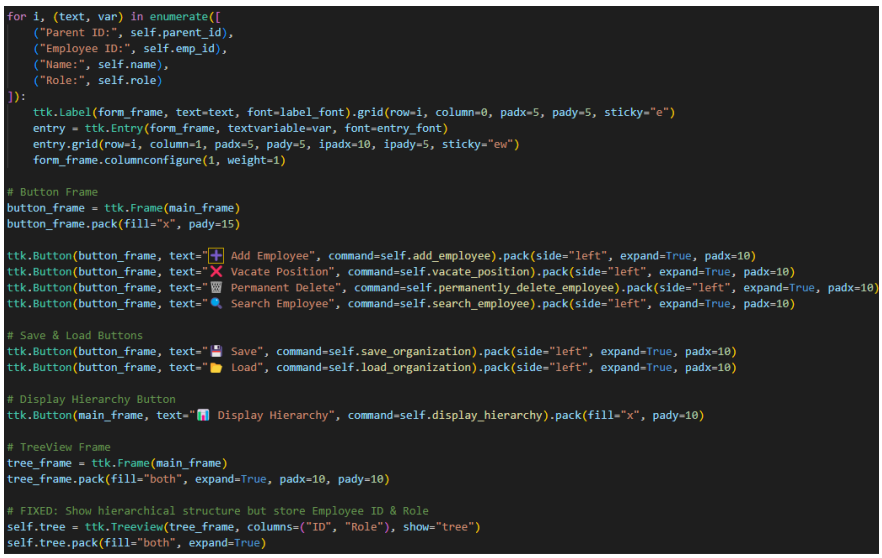
Figure 5.19

Prompt used after first AI code generated: Generate a better Python Tkinter GUI for an Organizational Chart Management System using a multi-level linked list data structure and the interface should include a treeview widget, input fields for employee ID, Name, Role & Parent ID. It should also include buttons for adding, vacating, deleting, searching, saving, and loading the hierarchy using JSON. The message boxes should also show invalid inputs and prevent duplicate employee IDs & handle cases where an organisation has no CEO.

After further prompt, the AI successfully managed to generate a working Tkinter GUI that integrates with backend logic.

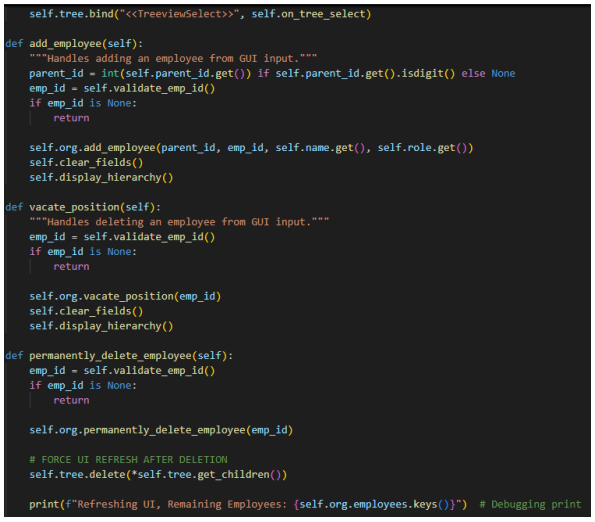
```python
import tkinter as tk
from tkinter import ttk, messagebox
from org import OrganizationChart  # Import your organization logic
import json  # Make sure to import json at the top
import os

class OrganizationGUI:
    def __init__(self, master):
        self.master = master
        self.master.title("Organizational Chart Management System")
        self.master.geometry("1200x1000")

        self.org = OrganizationChart()

        # Define input field
        self.parent_id = tk.StringVar()
        self.emp_id = tk.StringVar()
        self.name = tk.StringVar()
        self.role = tk.StringVar()

        # Styling
        style = ttk.Style()
        style.configure("TLabel", font=("Arial", 12))
        style.configure("TButton", font=("Arial", 12), padding=6)
        style.configure("Treeview", font=("Arial", 10), rowheight=20)

        # Main Frame
        main_frame = ttk.Frame(master, padding=20)
        main_frame.pack(fill="both", expand=True)

        # Form Frame
        form_frame = ttk.Frame(main_frame)
        form_frame.pack(fill="x", pady=10)

        label_font = ("Arial", 12, "bold")
        entry_font = ("Arial", 12)
```

Figure 5.20



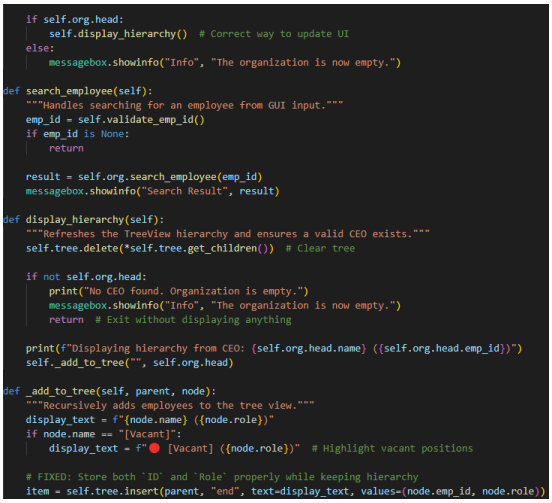
```python
for i, (text, var) in enumerate([
    ("Parent ID:", self.parent_id),
    ("Employee ID:", self.emp_id),
    ("Name:", self.name),
    ("Role:", self.role)
]):
    ttk.Label(form_frame, text=text, font=label_font).grid(row=i, column=0, padx=5, pady=5, sticky="e")
    entry = ttk.Entry(form_frame, textvariable=var, font=entry_font)
    entry.grid(row=i, column=1, padx=5, pady=5, ipadx=10, ipady=5, sticky="ew")
    form_frame.columnconfigure(1, weight=1)

# Button Frame
button_frame = ttk.Frame(main_frame)
button_frame.pack(fill="x", pady=15)

ttk.Button(button_frame, text="➕ Add Employee", command=self.add_employee).pack(side="left", expand=True, padx=10)
ttk.Button(button_frame, text="❌ Vacate Position", command=self.vacate_position).pack(side="left", expand=True, padx=10)
ttk.Button(button_frame, text="🗑️ Permanent Delete", command=self.permanently_delete_employee).pack(side="left", expand=True, padx=10)
ttk.Button(button_frame, text="🔍 Search Employee", command=self.search_employee).pack(side="left", expand=True, padx=10)

# Save & Load Buttons
ttk.Button(button_frame, text="💾 Save", command=self.save_organization).pack(side="left", expand=True, padx=10)
ttk.Button(button_frame, text="📂 Load", command=self.load_organization).pack(side="left", expand=True, padx=10)

# Display Hierarchy Button
ttk.Button(main_frame, text="📊 Display Hierarchy", command=self.display_hierarchy).pack(fill="x", pady=10)

# TreeView Frame
tree_frame = ttk.Frame(main_frame)
tree_frame.pack(fill="both", expand=True, padx=10, pady=10)

# FIXED: Show hierarchical structure but store Employee ID & Role
self.tree = ttk.Treeview(tree_frame, columns=("ID", "Role"), show="tree")
self.tree.pack(fill="both", expand=True)
```

Figure 5.21



```python
    self.tree.bind("<<TreeviewSelect>>", self.on_tree_select)

def add_employee(self):
    """Handles adding an employee from GUI input."""
    parent_id = int(self.parent_id.get()) if self.parent_id.get().isdigit() else None
    emp_id = self.validate_emp_id()
    if emp_id is None:
        return

    self.org.add_employee(parent_id, emp_id, self.name.get(), self.role.get())
    self.clear_fields()
    self.display_hierarchy()

def vacate_position(self):
    """Handles deleting an employee from GUI input."""
    emp_id = self.validate_emp_id()
    if emp_id is None:
        return

    self.org.vacate_position(emp_id)
    self.clear_fields()
    self.display_hierarchy()

def permanently_delete_employee(self):
    emp_id = self.validate_emp_id()
    if emp_id is None:
        return

    self.org.permanently_delete_employee(emp_id)

    # FORCE UI REFRESH AFTER DELETION
    self.tree.delete(*self.tree.get_children())

    print(f"Refreshing UI, Remaining Employees: {self.org.employees.keys()}")  # Debugging print
```

Figure 5.22



```python
    if self.org.head:
        self.display_hierarchy()  # Correct way to update UI
    else:
        messagebox.showinfo("Info", "The organization is now empty.")

def search_employee(self):
    """Handles searching for an employee from GUI input."""
    emp_id = self.validate_emp_id()
    if emp_id is None:
        return

    result = self.org.search_employee(emp_id)
    messagebox.showinfo("Search Result", result)

def display_hierarchy(self):
    """Refreshes the TreeView hierarchy and ensures a valid CEO exists."""
    self.tree.delete(*self.tree.get_children())  # Clear tree

    if not self.org.head:
        print("No CEO found. Organization is empty.")
        messagebox.showinfo("Info", "The organization is now empty.")
        return  # Exit without displaying anything

    print(f"Displaying hierarchy from CEO: {self.org.head.name} ({self.org.head.emp_id})")
    self._add_to_tree("", self.org.head)

def _add_to_tree(self, parent, node):
    """Recursively adds employees to the tree view."""
    display_text = f"{node.name} ({node.role})"
    if node.name == "[Vacant]":
        display_text = f"🔴 [Vacant] ({node.role})"  # Highlight vacant positions

    # FIXED: Store both `ID` and `Role` properly while keeping hierarchy
    item = self.tree.insert(parent, "end", text=display_text, values=(node.emp_id, node.role))
```

Figure 5.23

```
    # Recursively add all children
    child = node.child
    while child:
        self._add_to_tree(item, child)
        child = child.next

def clear_fields(self):
    """Clears all input fields."""
    self.parent_id.set("")
    self.emp_id.set("")
    self.name.set("")
    self.role.set("")

def validate_emp_id(self):
    """Validates and returns the employee ID as an integer, or None if invalid."""
    if not (emp_id := self.emp_id.get().strip()):
        messagebox.showerror("Error", "Please enter an Employee ID.")
        return None
    if not emp_id.isdigit():
        messagebox.showerror("Error", "Employee ID must be a number.")
        return None
    return int(emp_id)

def on_tree_select(self, event):
    """Handles filling input fields when selecting a tree node."""
    selected_item = self.tree.selection()
    if not selected_item:
        return

    item = selected_item[0]  # Get the selected item
    values = self.tree.item(item, "values")  # Extract stored values (ID, Role)
    name_with_role = self.tree.item(item, "text")  # Extract displayed text

    # FIXED: Handle missing values safely
```

Figure 5.24

```
    # FIXED: Handle missing values safely
    emp_id = values[0] if values else ""
    role = values[1] if len(values) > 1 else ""

    # Extract name from "Name (Role)" format safely
    name = name_with_role.rsplit(" (", 1)[0] if name_with_role else ""

    # Set input fields
    self.emp_id.set(emp_id)
    self.name.set(name)
    self.role.set(role)

    # Set Parent ID (if applicable)
    parent = self.tree.parent(item)
    if parent:
        parent_values = self.tree.item(parent, "values")
        self.parent_id.set(parent_values[0] if parent_values else "")
    else:
        self.parent_id.set("")

def save_organization(self):
    """Saves the organization chart to a JSON file."""
    filename = "organization_chart.json"
    try:
        self.org.save_to_json(filename)  # Use save_to_json() directly
        messagebox.showinfo("Success", f"Organization saved to {filename}")
    except Exception as e:
        messagebox.showerror("Error", f"Failed to save organization: {e}")

def load_organization(self):
    """Loads the organization chart from a JSON file."""
    filename = "organization_chart.json"

    if not os.path.exists(filename):  # Explicitly check file existence
```

Figure 5.25

```
    def load_organization(self):
        """Loads the organization chart from a JSON file."""
        filename = "organization_chart.json"

        if not os.path.exists(filename):  # Explicitly check file existence
            messagebox.showwarning("Warning", "No saved organization found.")
            return

        try:
            success = self.org.load_from_json(filename)  # Ensure this method returns a boolean
            if success:
                self.display_hierarchy()  # Refresh UI only if data was loaded
                messagebox.showinfo("Success", "Organization loaded successfully.")
            else:
                messagebox.showwarning("Warning", "File exists but contains no valid data.")
        except json.JSONDecodeError:  # Handle corrupt JSON
            messagebox.showerror("Error", "The file is corrupt or not in a valid format.")
        except Exception as e:
            messagebox.showerror("Error", f"Failed to load organization: {e}")

if __name__ == "__main__":
    root = tk.Tk()
    app = OrganizationGUI(root)
    root.mainloop()
```

Figure 5.26

After multiple corrections, the AI successfully generated a working Tkinter GUI (Figure 5.20 - 5.26) that integrates with the backend logic. Initially, the AI-generated code lacked features but after further prompting and refinements, it successfully built a functional Tkinter-based Organizational Chart Management System with proper data handling and user-interface updates.

Modifications Made to AI-Generated Code

1. *Multilinked List Structure & Helper Dictionary*

AI-Generated:

```
class EmployeeNode:
    def __init__(self, emp_id, name, role):
        self.emp_id = emp_id
        self.name = name
        self.role = role
        self.child = None
        self.next = None
```

Figure 5.27

AI had no parent pointers (causing it hard to move up the hierarchy), or last child pointers (without this adding multiple children is inefficient) & did not include a helper dictionary (self.employees) for fast lookups.

Modified Code:

```
class EmployeeNode:
    def __init__(self, emp_id, name, role):
        self.emp_id = emp_id
        self.name = name
        self.role = role
        self.next = None  # Pointer to next sibling
        self.child = None  # Pointer to first child
        self.parent = None  # Pointer to parent
        self.last_child = None  # Pointer to last child for fast insert

class OrganizationChart:
    def __init__(self):
        self.head = None  # Root node (CEO)
        self.employees = {}  # Dictionary for O(1) lookup
```

Figure 5.28

Added parent pointer for upward traversal.
Added last_child pointer to optimise insertions.
This helps to maintain the child-sibling relationship.
Added self.employees = {} under OrganizationChart for O(1) searching.

2. *Insertion Function (add_employee)*

AI-Generated:

```
def add_employee(self, parent_id, emp_id, name, role):
    """AI-generated insertion method (incorrect)."""

    new_employee = EmployeeNode(emp_id, name, role)

    if emp_id in self.employees:
        messagebox.showerror("Error", "Employee ID already exists.")
        return

    if parent_id not in self.employees:
        messagebox.showerror("Error", "Parent not found.")
        return

    # AI's incorrect linking logic
    parent = self.employees[parent_id]
    if parent.child is None:
        parent.child = new_employee
    else:
        current = parent.child
        while current.next:  # Traversing to the last child (inefficient)
            current = current.next
        current.next = new_employee  # Incorrectly linking the new employee he

    self.employees[emp_id] = new_employee
    messagebox.showinfo("Success", f"Employee {name} added under {parent_id}."
```

Figure 5.29

AI insertion does not handle CEO separately, allowing for multiple CEO. There are no parent pointers to track, no usage of last_child pointer (causing inefficient traversal to find last-child), and it incorrectly linked new employees (overwriting children, causing broken hierarchies), there wasn't handling for vacant positions.

Modified Code:

```
def add_employee(self, parent_id, emp_id, name, role):
    # Handle CEO addition separately
    if emp_id == 1:
        return self._handle_ceo_addition(name, role)

    # Ensure non-CEO employees have a valid parent
    if parent_id is None or parent_id not in self.employees:
        messagebox.showerror("Error", f"Parent ID {parent_id} not found.")
        return

    # Check if employee ID already exists
    if emp_id in self.employees:
        existing_employee = self.employees[emp_id]
        if existing_employee.name == "[Vacant]":
            existing_employee.name, existing_employee.role = name, role
            messagebox.showinfo("Success", f"Vacant position {emp_id} is now filled by {name}.")
        else:
            messagebox.showerror("Error", f"Employee ID {emp_id} already exists.")
        return

    # Add new employee
    new_employee = EmployeeNode(emp_id, name, role)
    self.employees[emp_id] = new_employee

    # Link to parent
    parent = self.employees[parent_id]
    new_employee.parent = parent

    if not parent.child:
        parent.child = new_employee
        parent.last_child = new_employee
    else:
        parent.last_child.next = new_employee
        parent.last_child = new_employee

    messagebox.showinfo("Success", f"Employee {emp_id} added under {parent_id}.")
```

Figure 5.30

Included proper CEO handling, ensuring only 1 exists.
Track **parent** pointers to help with traversal and deletion.
Using the **last_child** pointer to efficiently insert without traversing siblings.
Added **handling** for roles that are vacant allowing to refill empty positions for the new person getting hired to “takeover”

3. *Searching Function (search_employee)*

AI-Generated:

```
def search_employee(self, emp_id):
    """AI-generated search method (incorrect)."""

    for employee in self.employees.values():  # Inefficient O(n) search
        if employee.emp_id == emp_id:
            return f"ID: {employee.emp_id}, Name: {employee.name}, Role: {employee.role}"

    return "Employee not found."
```

Figure 5.31

AI search method uses a search loop rather than a dictionary lookup causing the search to be inefficient, and does not return the parent information causing it to be not informative.

Modified Code:

```
def search_employee(self, emp_id):
    employee = self.employees.get(emp_id)
    if employee:
        parent_id = employee.parent.emp_id if employee.parent else "None"
        return f"ID: {employee.emp_id}, Name: {employee.name}, Role: {employee.role}, Parent: {parent_id}"
    return "Employee not found."
```

Figure 5.32

Used **self.employees.get(emp_id)** for O(1) dictionary lookups, returns parent details and provides the full hierarchy context.

4. *Deletion (permanently_delete_employee)*

AI-Generated:

```
def delete_employee(self, emp_id):
    """AI Generated Deletion."""

    if emp_id not in self.employees:
        messagebox.showerror("Error", "Employee not found.")
        return

    employee = self.employees[emp_id]

    # Directly remove employee from the dictionary
    del self.employees[emp_id]

    # If employee had a parent, attempt to remove the reference
    if employee.parent and employee.parent.child == employee:
        employee.parent.child = employee.next  # Only updates the first child!

    # Doesn't reassign subordinates properly
    if employee.child:
        employee.child.parent = None  # Subordinates are orphaned!

    messagebox.showinfo("Success", f"Employee {emp_id} has been deleted.")
```

Figure 5.33

AI deletion method did not account for CEO deletion and treats all the employees the same, **causing the organisation structure** to be **broken** when the CEO is deleted. The parent-child reassignment is also incorrect as it only updates **parent.child = employee.next** this causes only the first child to be updated if the employee has multiple children causes the remaining sibling to be unlinked. Lastly, when deleting an employee

with children, it sets **employee.child.parent = None**. This unlinks the subordinates in the hierarchy, leaving them orphaned.

Modified Code:

```
def permanently_delete_employee(self, emp_id):
    """Permanently delete an employee and handle subordinates accordingly."""
    employee = self.employees.get(emp_id)
    if not employee:
        messagebox.showerror("Error", "Employee not found.")
        return

    if not messagebox.askyesno("Permanent Delete", f"Are you sure you want to permanently delete Employee {emp_id}?"):
        return

    # Handle CEO deletion separately
    if employee == self.head:
        self._handle_ceo_deletion(employee)
        return

    parent = employee.parent

    if employee.child:
        self._handle_subordinates_deletion(employee, parent)

    self._remove_from_parent(employee, parent)
    self._remove_employee(employee)

    messagebox.showinfo("Success", f"Employee {emp_id} has been permanently deleted.")
```

Figure 5.34

```
def _handle_ceo_deletion(self, ceo):
    print(f"🔴 Deleting CEO: {ceo.name} ({ceo.emp_id})")

    if messagebox.askyesno("Permanent Delete", "You are deleting the CEO. Do you want to delete the entire company?"):
        self.head = None
        self.employees.clear()
        print("✅ Entire organization deleted.")
        return

    if ceo.child:
        print(f"🔄 Promoting a new CEO from {ceo.name}'s children.")
        self._promote_new_ceo(ceo)
    else:
        self.head = None
        self.employees.clear()
        print("❌ No employees left after CEO deletion.")
```

Figure 5.35

```
def _handle_subordinates_deletion(self, employee, parent):
    """Handles subordinate deletion or reassignment when an employee is deleted."""
    if messagebox.askyesno("Delete Subordinates", f"Do you want to delete all subordinates of Employee {employee.emp_id}?"):
        self._delete_subtree(employee.child)
    else:
        self._reassign_subordinates(employee, parent)
```

Figure 5.36

```
def _remove_from_parent(self, employee, parent):
    if parent:
        print(f"🔴 Removing {employee.name} ({employee.emp_id}) from {parent.name} ({parent.emp_id})'s children")
        if parent.child == employee:
            parent.child = employee.next
            print(f"✅ {employee.name} ({employee.emp_id}) was the first child, new first child is {parent.child.name if parent.child else 'None'}")
        else:
            self._remove_from_sibling_list(parent.child, employee)
```

Figure 5.37

The modified code handles **missing employee** cases (Figure 5.34 to Figure 5.37), asks the user for confirmation before deletion, handles CEO deletion separately (_handle_ceo_deletion), reassigns subordinates properly (_handle_subordinates_deletion) & removes the employee from the parent list correctly (_remove_from_parent).

5. *Vacating (vacate_position)*

AI did not generate a vacate function at all causing improper handling for making an employee position vacant, the insertion function did not account for filling the vacant position.

Modified Code:

```
def vacate_position(self, emp_id):
    employee = self.employees.get(emp_id)
    if not employee:
        messagebox.showerror("Error", "Employee not found.")
        return
    employee.name = "[Vacant]"
    messagebox.showinfo("Success", f"Employee {emp_id} removed. Position is now vacant.")
```

Figure 5.38

Added a vacate function for employees to get replaced rather than getting deleted. This allows the employee to remain in the system keeping the hierarchy structure intact. The insertion function (as shown above under add_employee) will also be able to correctly refill the vacant position rather than create a new node.

6. *JSON Serialisation (_serialize_employee, _deserialize_employee)*

AI-Generated:

```python
def _serialize_employee(self, employee):
    """AI-generated incorrect serialization (missing last_child)."""
    if not employee:
        return None

    data = {
        "emp_id": employee.emp_id,
        "name": employee.name,
        "role": employee.role,
        "subordinates": []  # No last_child tracking
    }

    child = employee.child
    while child:
        data["subordinates"].append(self._serialize_employee(child))
        child = child.next

    return data
```

Figure 5.39

```python
def _deserialize_employee(self, data, parent=None):
    """AI-generated incorrect deserialization (missing last_child)."""
    if not data:
        return None

    emp = EmployeeNode(data["emp_id"], data["name"], data["role"])
    emp.parent = parent
    self.employees[emp.emp_id] = emp

    prev_child = None

    for child_data in data.get("subordinates", []):
        child = self._deserialize_employee(child_data, emp)
        if prev_child:
            prev_child.next = child  # Links children, but fails to t
        else:
            emp.child = child
        prev_child = child  # Updates last seen child

    return emp  # Last child reference is missing
```

Figure 5.40

The generated serialisation failed (Figure 5.39 to Figure 5.40) to maintain last_child while reconstructing the hierarchy. This lead to improper reconstruction, leading to broken links between employees.

Modified Code:

```python
def _deserialize_employee(self, data, parent=None):
    """Reconstructs an EmployeeNode from JSON data while maintaining `last_child`."""
    if not data:
        return None

    emp = EmployeeNode(data["emp_id"], data["name"], data["role"])
    emp.parent = parent
    self.employees[emp.emp_id] = emp

    prev_child = None
    first_child = None

    for child_data in data.get("subordinates", []):
        child = self._deserialize_employee(child_data, emp)
        if prev_child:
            prev_child.next = child
        else:
            first_child = child
        prev_child = child

    emp.child = first_child
    emp.last_child = prev_child  # Restore last_child reference

    return emp
```

Figure 5.41

```python
def _serialize_employee(self, employee):
    """Converts an EmployeeNode into a dictionary format for JSON storage."""
    if not employee:
        return None

    data = {
        "emp_id": employee.emp_id,
        "name": employee.name,
        "role": employee.role,
        "last_child": employee.last_child.emp_id if employee.last_child else None,  # Restore last_child
        "subordinates": []
    }

    child = employee.child
    while child:
        data["subordinates"].append(self._serialize_employee(child))
        child = child.next

    return data
```

Figure 5.42

The modified code restores last_child during serialisation (Figure 5.41) & deserialisation (Figure 5.42). It also ensures a correct traversal order in the multi-level linked list. It also maintains an accurate hierarchy structure, preventing any orphaned nodes.

Conclusion & Potential Improvements

The usage of AI to build the Organizational Chart Management System help to speed up the process at the beginning by generating the basis of the codes. However, it lacked essential optimisation and required multiple manual interventions and refinement. The AI-generated code was able to quickly provide a multi-level linked list structure for representation, generate basic insertion searching delete and serialisation functions, implement Tkinter GUI with a Treeview, and provide a visual representation of the hierarchy & its response time fast making refinements easier.

However, many issues came along as mentioned above like the initial data structure of the Multi-Level linked list lacked parent pointers and last_child optimisation and even forgot about the helper dictionary, insertion errors, deletion bugs, JSON serialisation issues, and Tkinter UI lacking features.

Some potential improvements for AI responses in future can be having better context awareness of what it outputting like recognising missing elements in its code and anticipating common edge cases. AI can also have a default option of using the most efficient data structure rather than using inefficient loops for searching. The generation of GUI features can be more interactive like generating event bindings and auto-have error messages to ensure better user experience. There can also be stronger debugging and test considerations when giving out the generation code to help the user debug.

While AI manages to build and generate a successful and fully functional system, software engineers/coders are still required to correct the logic and enhance the efficiency. This shows that there are still improvements needed to the AI to reduce the number of iterations needed to generate more and less prone free implementations from the start.

This application demonstrates and highlights the strengths and weaknesses of AI-generated code emphasising that AI is a powerful assistant but still requires a developer to refine and optimise the solution.

References

GeeksforGeeks. (2022). *Introduction to multi-linked list.* https://www.geeksforgeeks.org/introduction-to-multi-linked-list/

GeeksforGeeks. (2023). *Applications, advantages, and disadvantages of circular linked list.* https://www.geeksforgeeks.org/applications-advantages-and-disadvantages-of-circular-linked-list/

GeeksforGeeks. (2023). *Circular linked list meaning in DSA.* https://www.geeksforgeeks.org/circular-linked-list-meaning-in-dsa/

GeeksforGeeks. (2023). *Multilevel linked list.* https://www.geeksforgeeks.org/multilevel-linked-list/

GeeksforGeeks. (2023). *Applications, advantages and disadvantages of circular doubly linked list*. GeeksforGeeks. https://www.geeksforgeeks.org/applications-advantages-and-disadvantages-of-circular-doubly-linked-list/

GeeksforGeeks. (2024). *Difference between BFS and DFS.* https://www.geeksforgeeks.org/difference-between-bfs-and-dfs/

GeeksforGeeks. (2024). *Flatten a linked list with next and child pointers.* https://www.geeksforgeeks.org/flatten-a-linked-list-with-next-and-child-pointers/

GeeksforGeeks. (2024). *Introduction to circular doubly linked list.* https://www.geeksforgeeks.org/introduction-to-circular-doubly-linked-list/?ref=header_outind

GeeksforGeeks. (2024). *Skip list.* https://www.geeksforgeeks.org/skip-list/?ref=header_outind

GeeksforGeeks. (2024). *Types of linked list.* https://www.geeksforgeeks.org/types-of-linked-list/

HeyCoach. (2025). *Circular linked list operations.* https://blog.heycoach.in/circular-linked-list-operations-2/

GeeksforGeeks. (2025). *Circular linked list.*
https://www.geeksforgeeks.org/circular-linked-list/

HeyCoach. (2024). *Multi-level linked lists.*
https://blog.heycoach.in/multi-level-linked-lists/#:~:text=Traversing%20Multi%2Dlevel%20Linked%20Lists&text=This%20function%20visits%20each%20node,node%20at%20the%20current%20level.

OpenDSA. (2024). *Skip lists.*
https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/SkipList.html

Programiz. (n.d.). *Circular linked list.*
https://www.programiz.com/dsa/circular-linked-list

Quescol. (n.d.). *Doubly circular linked list*. Quescol.
https://quescol.com/data-structure/doubly-circular-linked-list

SparkCodeHub. (n.d.). *Data structure: Multi-level linked list.*
https://www.sparkcodehub.com/data-structure-multi-level-linked-list

Wikipedia. (2025). *Skip list.*
https://en.wikipedia.org/wiki/Skip_list

Wscubetech. (2025). Circular doubly linked list. Wscubetech.
https://www.wscubetech.com/resources/dsa/circular-doubly-linked-list